

UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ

DANIELA ARALDI

**ANÁLISE DAS REGRAS *SIGMA* GERADAS POR AGENTE AUTÔNOMO COM
RAG**

PATO BRANCO

2026

DANIELA ARALDI

**ANÁLISE DAS REGRAS *SIGMA* GERADAS POR AGENTE AUTÔNOMO COM
RAG**

Projeto de Trabalho de Conclusão de Curso de Graduação apresentado como requisito para obtenção do título de Bacharel em Engenharia de Computação do Curso de Bacharelado em Engenharia de Computação da Universidade Tecnológica Federal do Paraná.

Orientador: Prof. Me. Adriano Serckumecka

Coorientador: Prof. Dr. Jefferson Tales Oliva

PATO BRANCO

2026



[4.0 Internacional](https://creativecommons.org/licenses/by/4.0/)

Esta licença permite compartilhamento, remixe, adaptação e criação a partir do trabalho, mesmo para fins comerciais, desde que sejam atribuídos créditos ao(s) autor(es). Conteúdos elaborados por terceiros, citados e referenciados nesta obra não são cobertos pela licença.

RESUMO

Palavras-chave: regras sigma; engenharia de deteco; llm; rag; automao em ciberseguran.

ABSTRACT

Keywords: sigma rules; detection engineering; llm; rag; cybersecurity automation.

LISTA DE QUADROS

Quadro 1 – Modelos de linguagem utilizados no trabalho.	34
Quadro 2 – <i>Frameworks</i> , banco vetorial e ambiente de desenvolvimento.	35
Quadro 3 – Bibliotecas de validação, coleta de dados e análise estatística.	36
Quadro 4 – <i>Hardware</i> utilizado no desenvolvimento do trabalho.	36

LISTA DE ABREVIATURAS E SIGLAS

Siglas

API	<i>Application Programming Interface</i> , em português Interface de Programação de Aplicações
BGL	<i>BlueGene/L</i>
CAPEC	Common Attack Pattern Enumerations and Classifications
CISA	Agência de Segurança de Infraestrutura e Cibernética, do inglês <i>Cybersecurity and Infrastructure Security Agency</i>
CNN	Rede Neural Convolucional, do inglês <i>Convolutional Neural Network</i>
CoT	<i>Chain-of-Thought</i>
CoT-R	<i>Chain-of-Thought + Reflection</i>
CVE	Vulnerabilidades e Exposições Comuns, do inglês <i>Common Vulnerabilities and Exposures</i>
CVSS	Sistema de Pontuação de Vulnerabilidades Comuns, do inglês <i>Common Vulnerability Scoring System</i>
CWE	Common Weakness Enumeration, do inglês <i>Common Weakness Enumeration</i>
DVWA	Damn Vulnerable Web Application, do inglês <i>Damn Vulnerable Web Application</i>
EPP	<i>Endpoint Protection Platforms</i>
GDPR	Regulamento Geral de Proteção de Dados, do inglês <i>General Data Protection Regulation</i>
GPU	<i>Graphics Processing Unit</i>
IA	Inteligência Artificial, do inglês <i>Artificial Intelligence</i>
IDS	<i>Intrusion Detection System</i>
IOC	Indicador de Comprometimento, do inglês <i>Indicator of Compromise</i>
IPS	<i>Intrusion Prevention System</i>
IR	<i>Intermediate Representation</i>

KQL	<i>Kusto Query Language</i>
LLM	Modelo de linguagem de grande escala, do inglês, <i>Large Language Model</i>
ML	Aprendizado de Máquina, do inglês <i>Machine Learning</i>
NIST	Instituto Nacional de Padrões e Tecnologia, do inglês <i>National Institute of Standards and Technology</i>
NVD	Banco Nacional de Vulnerabilidades, do inglês <i>National Vulnerability Database</i>
OSCTI	<i>Open-Source Cyber Threat Intelligence</i>
PCI-DSS	Padrão de Segurança de Dados da Indústria de Cartões de Pagamento, do inglês <i>Payment Card Industry Data Security Standard</i>
PLN	Processamento de Linguagem Natural, do inglês <i>Natural Language Processing</i>
QA	<i>Question Answering</i>
RAG	<i>Retrieval-Augmented Generation</i> , em português, Geração Aumentada por Recuperação
RNN	<i>Recurrent Neural Networks</i> , Redes Neurais Recorrentes.
SIEM	Sistema de Gerenciamento de Informações e Eventos de Segurança, do inglês <i>Security Information and Event Management</i>
SOC	Centro de Operações de Segurança, do inglês <i>Security Operations Center</i>
SPL	<i>Splunk Processing Language</i>
TF-IDF	Frequência de Termo–Frequência Inversa de Documento, do inglês <i>Term Frequency–Inverse Document Frequency</i>
TTP	<i>Tactics, Techniques, and Procedures</i> — em português, Táticas, Técnicas e Procedimentos
URL	<i>Uniform Resource Locators</i>

SUMÁRIO

1	INTRODUÇÃO	8
1.1	Motivação	8
1.2	Objetivos	10
1.2.1	Objetivo geral	10
1.2.2	Objetivos específicos	10
1.3	Estrutura do trabalho	11
2	REFERENCIAL TEÓRICO	12
2.1	Segurança da Informação e SOC	12
2.1.1	SIEM	12
2.1.2	SIEMs Inteligentes	14
2.2	Bases de Conhecimento sobre Vulnerabilidades e Técnicas Adversárias	16
2.2.1	CVE e CWE	16
2.2.2	MITRE ATT&CK	17
2.2.3	NVD e CVSS	17
2.3	Regras <i>Sigma</i>	18
2.3.1	Sigma e a mitigação de ameaças	19
2.3.2	Estrutura	19
2.4	Processamento de Linguagem Natural	21
2.4.1	Modelos de Linguagem de Grande Escala (LLMs)	22
2.4.2	Arquitetura <i>Transformer</i>	23
2.4.3	Engenharia de <i>Prompt</i>	24
2.5	Geração Aumentada por Recuperação (RAG)	25
2.6	Agentes Autônomos	25
3	TRABALHOS RELACIONADOS	26
3.1	<i>LLMCloudHunter</i>	26
3.2	<i>RulePilot</i>	27
3.3	GRIDAI	29
3.4	Aprendizado de máquina aplicado a SIEMs e análise de <i>logs</i>	30
3.5	Posicionamento do Trabalho Proposto	31
4	MATERIAIS E MÉTODOS	34

4.1	Materiais	34
4.1.1	Base de Dados	37
4.1.1.1	<u>Organização dos dados em disco</u>	38
4.2	Métodos	38
4.2.1	Seleção dos dados	38
4.2.2	Estratégias de geração das regras	40
4.2.2.1	<u>Regras originais do <i>SigmaHQ</i> (Estágio 1)</u>	40
4.2.2.2	<u>Geração zero-shot com s comerciais (Estágio 2)</u>	41
4.2.2.3	<u>Geração com <i>prompt engineering</i> (Estágio 3)</u>	41
4.2.2.4	<u>Geração via agente com RAG (Estágio 4)</u>	42
4.2.3	Desenvolvimento do agente	44
4.2.3.1	<u>Visão geral da arquitetura</u>	44
4.2.3.2	<u>Máquina de estados (<i>LangGraph</i>)</u>	46
4.2.3.3	<u>Ferramentas externas</u>	48
4.2.3.4	<u>RAG</u>	48
4.2.3.5	<u>Geração da regra</u>	48
4.2.3.6	<u>Validação sintática iterativa e laço de correção</u>	48
4.2.3.7	<u>Avaliação semântica por LLL-as-a-judge</u>	48
4.2.4	Métricas de Avaliação	48
4.2.4.1	<u>Determinísticas</u>	48
4.2.4.2	<u>Similaridade</u>	48
4.2.4.3	<u>Comparação entre estágios</u>	48
5	RESULTADOS	49
5.1	Discussões (opcional)	49
5.2	Trabalhos futuros	49
6	CONCLUSÃO	50
	REFERÊNCIAS	51
	GLOSSÁRIO	56

1 INTRODUÇÃO

1.1 Motivação

A segurança cibernética contemporânea sustenta a estabilidade de serviços essenciais — como saúde, energia, finanças e administração pública, enquanto enfrenta uma pressão crescente decorrente tanto do aumento no volume de ameaças quanto da complexidade das infraestruturas tecnológicas modernas. Os ataques cibernéticos alimentam um ciclo contínuo de exploração de vulnerabilidades, comprometendo desde a integridade de dados sensíveis até a continuidade operacional de organizações inteiras. Esse ciclo demanda abordagens de defesa cada vez mais ágeis e adaptativas, capazes de acompanhar a sofisticação dos ataques e a expansão de infraestruturas tecnológicas heterogêneas (Shah *et al.*, 2022).

Por infraestrutura heterogênea, entende-se um ambiente composto por diferentes tecnologias, dispositivos, sistemas operacionais, plataformas e arquiteturas que coexistem e interagem entre si. Tal heterogeneidade amplia a complexidade de gestão e de proteção, pois cada componente possui vulnerabilidades específicas e demanda estratégias distintas de defesa. Nesse contexto, um único ponto fraco pode comprometer todo o ecossistema, o que reforça a necessidade de uma defesa cibernética inovadora e adaptativa (Shah *et al.*, 2022; PALO ALTO NETWORKS, 2025; EXABEAM, 2025).

A dimensão desse desafio pode ser quantificada pelo ritmo de divulgação de vulnerabilidades. Só em 2025 foram publicados aproximadamente 48 mil registros de Vulnerabilidades e Exposições Comuns, do inglês *Common Vulnerabilities and Exposures* (CVE)s — uma média de cerca de 133 novas vulnerabilidades por dia e um crescimento de aproximadamente 21% em relação a 2024 (Gamblin, 2026). O volume tornou-se de tal forma insustentável que, em abril de 2026, o Instituto Nacional de Padrões e Tecnologia, do inglês *National Institute of Standards and Technology* (NIST) anunciou que seu Banco Nacional de Vulnerabilidades, do inglês *National Vulnerability Database* (NVD) deixaria de enriquecer todos os CVEs recebidos, passando a priorizar apenas uma fração deles, em razão de um aumento de 263% nas submissões entre 2020 e 2025 (National Institute of Standards and Technology, 2026). Este cenário torna evidente que o esforço manual de catalogação e análise de ameaças já não acompanha a velocidade com que elas surgem.

A urgência dessa adaptação se tornou ainda mais evidente diante da diminuição progressiva do intervalo entre a divulgação de uma vulnerabilidade e sua exploração. Os *zero-days* — ataques que exploram falhas ainda desconhecidas do desenvolvedor, sempre constituíram um caso particular em que a exploração precede a correção; o que se observa nos últimos anos, contudo, é a inversão da média desse intervalo. Relatórios de inteligência de ameaças documentam essa trajetória de forma quantitativa: o tempo médio para exploração (do inglês *time-to-exploit*) caiu de aproximadamente 63 dias em 2018 para cerca de 32 dias em 2023, cruzou o zero em 2024 e atingiu, em 2025, uma estimativa de -7 dias (Mandiant, 2026). Em

outras palavras, a exploração antecipando a divulgação deixou de ser exceção, característica dos *zero-days*, e passou a representar o comportamento médio do ecossistema de ameaças. A consequência estratégica é direta, pois quando a exploração precede o *patch*, a aplicação de correções deixa de ser o controle decisivo, e a capacidade de detectar a atividade maliciosa em tempo hábil passa a ser o fator determinante para conter o ataque (Rapid7 Labs, 2026). A velocidade de detecção se tornou a linha de frente da defesa.

É precisamente neste contexto que se insere a engenharia de detecção, do inglês *detection engineering* — área da cibersegurança dedicada ao projeto, à validação e à manutenção sistemática das lógicas de detecção que alimentam os mecanismos de defesa. Os Sistema de Gerenciamento de Informações e Eventos de Segurança, do inglês *Security Information and Event Management* (SIEM)s, consolidaram-se como pilares dessa atividade, correlacionando grandes volumes de registros automáticos de eventos (*logs*) gerados continuamente por sistemas, aplicações, redes e dispositivos. Assim como os *firewalls*, os *Intrusion Detection System* (IDS)s e os *Intrusion Prevention System* (IPS), os SIEMs operam fundamentalmente com base em regras de detecção. Tais regras, contudo, nem sempre acompanham a evolução das técnicas maliciosas, exigindo atualização constante (Shah *et al.*, 2022; Podzins; Romanovs, 2019).

O gargalo da engenharia de detecção, entretanto, não está apenas na defasagem das regras, mas no próprio esforço humano de produzi-las e mantê-las. Pesquisas recentes do setor apontam que a fadiga de alertas, do inglês *alert fatigue*, figura entre as principais preocupações dos Centro de Operações de Segurança, do inglês *Security Operations Center* (SOC)s, e que os falsos positivos constituem o desafio mais citado na detecção de ameaças, sobrecarregando equipes já afetadas pela escassez de profissionais qualificados (SANS Institute, 2025). Cada regra mal calibrada gera ruído; cada ruído consome o tempo de analistas que precisariam estar investigando ameaças reais. Reduzir o custo humano de produzir regras de detecção precisas e atualizadas é um problema substancial da defesa cibernética moderna.

Dentre as iniciativas que buscam padronizar e acelerar essa produção, destacam-se as regras *Sigma*: um formato aberto e descritivo, escrito em *YAML* (linguagem de serialização de dados legível e estruturada), voltado à definição de lógicas de detecção independentes de plataforma. Por serem convertíveis para linguagens nativas de diferentes SIEMs, como *Splunk Processing Language* (SPL) ou *Elasticsearch Query*, as regras *Sigma* favorecem a colaboração entre equipes e a portabilidade das detecções (SIGMAHQ, 2025a). No entanto, sua criação e manutenção manuais permanecem lentas e propensas a lacunas, sobretudo diante de ameaças emergentes como um *ransomware* (tipo de *malware* que sequestra dados ou sistemas e exige resgate) ou um *zero-day exploit*, ataque que explora uma falha de segurança ainda desconhecida do desenvolvedor e para a qual não há correção disponível (SIGMAHQ, 2025a; EXABEAM, 2025).

Diante deste conjunto de pressões — o volume insustentável de vulnerabilidades, a compressão do tempo de exploração e o custo humano da engenharia de detecção manual, a aplicação de técnicas de Inteligência Artificial, do inglês *Artificial Intelligence* (IA) na geração

automatizada de regras apresenta-se como uma direção promissora, capaz de reduzir o intervalo entre a divulgação de uma ameaça e sua cobertura defensiva, preservando a qualidade técnica das regras produzidas. O estudo de Lempinen, Pyyny e Juntunen (2023) apud Hasanov *et al.* (2024a) demonstrou que a integração entre LLM, e o SIEM *Wazuh*¹ resultou em eficácia na análise de *logs*, na detecção de padrões de ameaças e na automação de respostas de segurança, beneficiando sobretudo usuários com conhecimento limitado na área. Tal evidência sugere que LLMs podem ser adaptados para apoiar a geração de regras *Sigma* a partir de dados de ameaças.

Sendo assim, este trabalho propõe a implementação de um agente autônomo capaz de automatizar a criação de regras *Sigma* a partir de referências externas de inteligência de ameaças — tipicamente CVEs, Indicador de Comprometimento, do inglês *Indicator of Compromise* (IOC)s, *logs* de incidentes e notícias de vulnerabilidades recém-divulgadas, acessadas durante o processo de geração. A proposta é concebida como ferramenta de apoio à engenharia de detecção, na qual o analista permanece como instância final de validação no fluxo operacional, ainda que a presente investigação se concentre na avaliação da geração automática propriamente dita. O agente combina recuperação aumentada por geração (*Retrieval-Augmented Generation*, em português, Geração Aumentada por Recuperação (RAG)), sobre um repositório consolidado de regras *Sigma*, consulta a fontes externas via busca na *web* e validação iterativa (sintática e semântica) das regras produzidas por um Modelo de Linguagem de Grande Escala. Ao final, os resultados obtidos por diferentes estratégias de geração, com complexidade arquitetural crescente, são comparados de modo a avaliar o ganho efetivo proporcionado pela arquitetura agêntica proposta.

1.2 Objetivos

1.2.1 Objetivo geral

Desenvolver um agente autônomo capaz de gerar regras *Sigma* de forma automatizada a partir de fontes diversificadas de inteligência de ameaças — CVEs, (IOCs), *logs* de incidentes e notícias de vulnerabilidades recém-divulgadas, com o propósito de acelerar a operacionalização de detecções em SIEMs e, conseqüentemente, reduzir o intervalo entre a divulgação de uma ameaça e a sua cobertura defensiva.

1.2.2 Objetivos específicos

- Replicar regras *Sigma* a partir de LLMs;

¹ O *Wazuh* é uma solução de segurança de código aberto com funcionalidades de *Host-based Intrusion Detection System* (HIDS) e SIEM (Wazuh, Inc., 2024).

- Gerar regras *Sigma* a partir do agente autônomo construído;
- Desenvolver um agente autônomo para a geração de regras *Sigma*;
- Comparar os resultados dos métodos utilizados;

1.3 Estrutura do trabalho

Este trabalho possui cinco capítulos, organizados de forma a conduzir o leitor do embasamento teórico à abordagem metodológica que será adotada. No primeiro capítulo, apresenta-se a introdução do tema, defendendo a motivação, os objetivos e a justificativa do estudo. O capítulo 2 disserta sobre conceitos necessários para se compreender o assunto do trabalho: sistemas SIEM, regras *Sigma* e LLMs — o que são, como se configuram na segurança da informação e como se relacionam entre si diante da mitigação de ameaças. Já o capítulo 3 discute trabalhos relacionados, oferecendo uma visão crítica sobre pesquisas e soluções correlatas. O capítulo 4 descreve os materiais e os métodos utilizados, detalhando as etapas do desenvolvimento da proposta. No capítulo 5 estão contidos os Resultados ... ajeitar

2 REFERENCIAL TEÓRICO

Este capítulo apresenta os fundamentos teóricos que sustentam o desenvolvimento do agente proposto. São discutidos os conceitos de SOC e SIEM na área de segurança da informação, as bases de conhecimento sobre vulnerabilidades e técnicas adversárias, o padrão *Sigma* para regras de detecção, os modelos de linguagem de grande escala (LLMs), a geração aumentada por recuperação (RAG) e os agentes autônomos como paradigma de orquestração de LLMs.

2.1 Segurança da Informação e SOC

Em organizações de médio e grande porte a defesa cibernética não costuma ser tarefa de uma única ferramenta. O volume e a heterogeneidade dos eventos gerados por servidores, *firewalls*, sistemas de detecção de intrusão (IDS), aplicações em nuvem e dispositivos de usuários tornam inviável a inspeção manual contínua, exigindo a composição de equipes e infraestruturas dedicadas ao monitoramento, à detecção e à resposta a incidentes (Vielberth *et al.*, 2020; Shah *et al.*, 2022). É neste contexto que surgem os SOC, unidades especializadas que centralizam pessoas, processos e tecnologias para sustentar a postura defensiva de uma organização em tempo integral.

Segundo Vielberth *et al.* (2020), um SOC bem estruturado articula três pilares interdependentes: analistas humanos com diferentes níveis de experiência, procedimentos formalizados de triagem e resposta e uma camada tecnológica capaz de coletar, correlacionar e priorizar evidências de ameaças. A operação típica se organiza em níveis funcionais, em que no *Tier 1* os analistas iniciais executam a triagem dos alertas, no *Tier 2* os investigadores intermediários conduzem a análise de incidentes e no *Tier 3* os especialistas de nível mais alto realizam *Threat Hunting* e perícia forense (Tariq *et al.*, 2025). O componente tecnológico central que sustenta essa cadeia é o sistema SIEM, descrito na próxima seção.

2.1.1 SIEM

Um SIEM atua como o núcleo de inteligência operacional em ambientes de cibersegurança, agregando e correlacionando *logs* – registros automáticos de eventos gerados por sistemas, aplicações, redes e dispositivos, provenientes de toda a infraestrutura de Tecnologia da Informação. Como já citado anteriormente, as fontes típicas incluem *firewalls*, sistemas de detecção e prevenção de intrusão (IDS/IPS), servidores de aplicação e de banco de dados, controladores de domínio, *endpoints* e serviços em nuvem (González-Granadillo; González-Zarzosa; Diaz, 2021). Seu propósito reside na capacidade de correlacionar eventos aparentemente desconexos e identificar padrões que ferramentas isoladas não o fazem.

Convém distinguir o SIEM de outras soluções pontuais de defesa, como antivírus, *firewalls* ou plataformas de proteção de *endpoint* (EPP). Tais soluções operam de forma localizada – protegendo um *host*, um perímetro de rede ou um vetor específico, e geram alertas específicos de seu escopo. O SIEM, por outro lado, é uma camada agregadora: ele consome os *logs* produzidos por essas ferramentas, normaliza formatos heterogêneos para um esquema comum e aplica regras de correlação capazes de identificar, na conjunção de múltiplos eventos, padrões de ataque que permaneceriam imperceptíveis em uma análise isolada (González-Granadillo; González-Zarzosa; Diaz, 2021; Sheeraz *et al.*, 2024). Conforme destacam Podzins e Romanovs (2019), essa contextualização é decisiva em ações automatizadas contra ameaças complexas, como ataques *zero-day*, em que, por exemplo, a correlação entre uma tentativa de força bruta no *Active Directory* (serviço de diretório da *Microsoft* responsável pelo gerenciamento de identidades, recursos e permissões da rede interna) e um tráfego suspeito no *firewall* pode disparar uma resposta imediata.

Nos SOC, o SIEM assume papel operacional indispensável: consolida alertas de diferentes fontes e formatos, prioriza incidentes com base na criticidade e orquestra fluxos de resposta (*workflows*). Sem essa camada, os analistas perdem a capacidade de detectar fases avançadas de ataque, como movimento lateral e escalção de privilégios – etapas em que o invasor se desloca pelos sistemas do alvo acumulando privilégios até atingir o nível administrativo (Podzins; Romanovs, 2019). A ausência desse tipo de visão integrada está associada a violações de longa duração, como foi o caso da rede mundial de hotéis *Starwood*, em que a intrusão permaneceu não identificada por aproximadamente quatro anos e resultou no vazamento de dados de cerca de 383 milhões de hóspedes ao redor do mundo (Shepardson, 2019). Essa visão unificada transforma o SIEM no "sistema nervoso" do SOC, permitindo que analistas investiguem ameaças sistêmicas e não apenas alertas isolados.

A eficácia de um SOC, porém, está intrinsecamente ligada à robustez do SIEM e à calibração das suas regras de detecção. Regras mal calibradas geram excesso de falsos positivos e instâncias não otimizadas podem produzir mais de mil alertas diários, sobrecarregando as equipes – segundo Podzins e Romanovs (2019), um analista isolado consegue investigar com profundidade entre 5 e 15 alertas por dia, dependendo da criticidade. Esse desequilíbrio configura o fenômeno conhecido como *alert fatigue*, ou fadiga de alertas: a exposição contínua a grandes volumes de notificações de baixo valor leva os analistas à dessensibilização, ao prolongamento do tempo de resposta e, em última instância, ao *burnout* profissional, com impactos diretos na qualidade da defesa (Tariq *et al.*, 2025). A literatura recente identifica a fadiga de alertas como um dos principais desafios contemporâneos dos SOC e, justamente por isso, motiva a busca por mecanismos automatizados de triagem, correlação e geração de regras mais precisas – preocupação central do presente trabalho.

A importância estratégica do SIEM também se ancora na sua capacidade de reduzir o tempo de detecção de ameaças. Podzins e Romanovs (2019) citam o relatório de violações da *Verizon* de 2018, que apontava cerca de 68% das violações sendo descobertas meses ou

mais após o ataque inicial (Business, 2018); já o relatório de 2025 informa que o tempo médio de remediação de vulnerabilidades em dispositivos de borda (especialmente falhas de *zero-day*) é de 32 dias (Business, 2025). Essa lacuna pode ser substancialmente reduzida com um SIEM bem implementado, em virtude da correlação de eventos em tempo real. Adicionalmente, a plataforma SIEM serve de alicerce para a evolução tecnológica do SOC: é sobre ela que se integram componentes mais recentes, como módulos de inteligência artificial e aprendizado de máquina, que ampliam a detecção de ameaças sutis e habilitam a automação de respostas (Tendikov *et al.*, 2024).

Do ponto de vista econômico, embora o SIEM envolva custos elevados – com licenças baseadas em volume de *logs*, infraestrutura dedicada e equipe especializada em regime ininterrupto, o investimento se justifica diante da magnitude das perdas associadas a incidentes cibernéticos. Em 2019, a *Accenture* projetava perdas globais da ordem de US\$ 5,2 trilhões para os 5 anos seguintes (Accenture, 2019 apud Podzins; Romanovs, 2019), tendência que se confirmou nos anos posteriores: o relatório anual da IBM de 2024 ((IBM Security, 2024)) registrou um custo médio global de US\$ 4,88 milhões por violação de dados, com aumento de 10% em relação ao ano anterior. O último, de 2025, no entanto, trouxe uma pequena reviravolta: o custo médio global caiu 9% em relação a 2024, motivado por melhorias na identificação e contenção de incidentes (IBM Security, 2025). Para organizações com dados sensíveis ou sujeitas a regulações como o Regulamento Geral de Proteção de Dados, do inglês *General Data Protection Regulation* (GDPR), da União Europeia, ou o PCI-DSS – norma globalmente reconhecida para a segurança de dados de cartões de pagamento, o SIEM deixa de ser uma opção e torna-se requisito operacional. Em síntese, o SIEM mantém-se como pilar irrevogável da segurança corporativa, não apenas pela sua tecnologia isolada, mas por construir o ecossistema integrado de detecção e resposta sobre o qual se constroem as defesas modernas.

2.1.2 SIEMs Inteligentes

Apesar de sua centralidade operacional, o o sistema SIEM tradicional opera majoritariamente sobre regras estáticas – definições escritas por especialistas que descrevem condições de alertas a partir de padrões conhecidos. Essa abordagem apresenta duas limitações sistêmicas: a dificuldade em detectar ameaças cuja assinatura ainda não foi formalizada, como ataques *zero-day*, e a alta taxa de falsos positivos quando as regras não acompanham a dinâmica do ambiente protegido (González-Granadillo; González-Zarzosa; Diaz, 2021; Nurusheva *et al.*, 2024). González-Granadillo, González-Zarzosa e Diaz (2021) observam, especificamente, que pouquíssimas soluções de SIEM dispõem de mecanismos de correlação avançada capazes de realizar uma análise histórica e de desvios necessária, por exemplo, para a detecção posterior de ataques zero-day. Portanto, é a partir da integração de técnicas de aprendizado de máquina Aprendizado de Máquina, do inglês *Machine Learning* (ML) e IA, buscando superar tais limitações, que se originam os chamados SIEMs inteligentes.

De acordo com Saddi *et al.* (2024), a IA generativa é capaz de produzir dados sintéticos e simular cenários de ataque – replicando, por exemplo, a atuação de um invasor ou a propagação de *malware*, o que permite tanto treinar modelos de detecção quanto avaliar defesas antes de sua implantação, sustentando uma postura proativa de identificação de ameaças emergentes. Essa capacidade é amplificada por algoritmos de ML que analisam padrões contextuais em grandes volumes de *logs*, correlacionam eventos aparentemente desconexos – por exemplo, tráfego anômalo combinado com tentativas de força bruta, e identificam comportamentos compatíveis com técnicas adversárias documentadas. Tendikov *et al.* (2024) reforçam que a integração de IA e ML em SIEMs aprimora a correlação de grandes volumes de eventos em tempo real e permite que esses sistemas aprendam a partir de dados históricos e se adaptem dinamicamente a ameaças emergentes. Esse aprimoramento incide diretamente sobre a fadiga dos analistas discutida na seção anterior: Tariq *et al.* (2025) documentam que mecanismos de automação e triagem assistida por ML reduzem a carga cognitiva associada ao alto volume de alertas e ao excesso de falsos positivos, principais causas do esgotamento nos SOCs.

Nurusheva *et al.* (2024) demonstram que algoritmos de detecção de anomalias, análise preditiva e limiares adaptativos permitem identificar atividades sutis que escapariam das regras estáticas ou à análise manual, e propõem um sistema autoadaptativo, capaz de identificar ameaças inéditas em tempo real sem depender de regras predefinidas. Um aspecto central desse paradigma é a possibilidade de reduzir a dependência de bases previamente rotuladas ¹: como destacado por Tariq *et al.* (2025), algoritmos não supervisionados conseguem modelar o comportamento típico de uma rede e detectar anomalias sem a necessidade de conjuntos de dados rotulados, o que os torna particularmente eficazes na identificação de ameaças novas – embora ao custo de uma taxa de falsos positivos superior à dos métodos supervisionados. Do ponto de vista conceitual, esse comportamento decorre das próprias características do aprendizado de máquina, cuja capacidade de aprender padrões a partir de dados, sem programação explícita, fundamenta tanto as abordagens supervisionadas quanto as não-supervisionadas (Jordan; Mitchell, 2015). Ao combinar esses dois tipos de modelo, o SIEM com ML ajusta-se continuamente às mudanças no ambiente e antecipa novos vetores de ataque (Sheeraz *et al.*, 2024).

Por fim, compreende-se que depender exclusivamente de analistas humanos é inviável diante do crescimento exponencial dos dados e da sofisticação das ameaças. A incorporação de IA e ML em SIEMs representa, portanto, um avanço significativo da cibersegurança, conferindo precisão, adaptabilidade e automação a um cenário cada vez mais dinâmico e hostil. Esse movimento, no entanto, ainda enfrenta desafios relevantes, sobretudo no que tange à interpretabilidade dos modelos, à dependência de dados de qualidade para o treinamento e à necessidade de regras formais que estabeleçam o vocabulário comum entre as diferentes ferramentas. Seguindo este último ponto, nas próximas seções se discutirá sobre a efetividade

¹ Uma base rotulada (do inglês *labeled dataset*) condiz a uma coleção de pares entrada-saída, onde cada amostra de entrada (por exemplo, linha de *log* ou fluxo de rede) recebeu uma anotação confiável sobre o que se é de fato

de qualquer SIEM; sendo inteligente ou não, depende de regras de detecção formalizadas em padrões abertos e interoperáveis, ancoradas em bases consolidadas de conhecimento sobre vulnerabilidades e técnicas adversárias.

2.2 Bases de Conhecimento sobre Vulnerabilidades e Técnicas Adversárias

A construção de regras de detecção em SIEMs depende de um vocabulário comum para descrever vulnerabilidades, fraquezas e comportamentos adversários. Esse vocabulário é fornecido por bases abertas, mantidas por organizações de pesquisa e agências governamentais, que catalogam de forma sistemática os elementos de interesse para a defesa cibernética. Sem esses padrões, regras escritas por diferentes equipes seriam incomparáveis, e a interoperabilidade entre ferramentas seria inviável (Roy *et al.*, 2023). O agente proposto neste trabalho consulta quatro dessas bases ao longo de sua execução: CVE, Common Weakness Enumeration, do inglês *Common Weakness Enumeration* (CWE), NVD e MITRE ATT&CK. As subseções a seguir apresentam cada uma delas e justificam seu papel na geração das regras *Sigma*.

2.2.1 CVE e CWE

O CVE é um catálogo público de vulnerabilidades, operado pelo MITRE desde 1999 sob patrocínio da Agência de Segurança de Infraestrutura e Cibernética, do inglês *Cybersecurity and Infrastructure Security Agency* (CISA) — agência norte-americana de segurança cibernética (The MITRE Corporation, 2024)). Cada vulnerabilidade divulgada recebe um identificador único no formato CVE-AAAA-NNNN, em que AAAA é o ano da publicação e NNNN um número sequencial. Em 2024, o catálogo já ultrapassava 250 mil registros (The MITRE Corporation, 2025b). A principal função do CVE é eliminar ambiguidade: quando duas ferramentas mencionam "CVE-2021-44228", há certeza de que ambas se referem à mesma falha — aqui, a vulnerabilidade *Log4Shell*, na biblioteca *Apache Log4j*. Neste trabalho, o CVE atua como uma das entradas possíveis para o agente: quando o usuário fornece um identificador, ou quando a regra parte de uma descrição textual que o menciona, o agente consulta o catálogo para extrair contexto descritivo.

O CWE, também mantido pelo MITRE, opera em outro nível de abstração. Enquanto o CVE cataloga vulnerabilidades específicas em produtos identificáveis, o CWE classifica as categorias gerais de fraqueza subjacentes — *buffer overflow*, falhas de validação de entrada, exposição de dados sensíveis e centenas de outras. Uma vulnerabilidade CVE costuma estar associada a um ou mais CWEs que descrevem a natureza do problema (The MITRE Corporation, 2025a).

Para a geração de regras, essa abstração é valiosa: padrões CWE generalizam-se para famílias inteiras de vulnerabilidades, incluindo variantes ainda não catalogadas. O agente pro-

posto obtém os CWEs associados a um CVE por meio da consulta ao NVD, descrita na **subseção x**.

2.2.2 MITRE ATT&CK

Em complemento aos catálogos de vulnerabilidades e fraquezas, o MITRE ATT&CK (*Adversarial Tactics, Techniques, and Common Knowledge*) fornece uma taxonomia de *Tactics, Techniques, and Procedures* — em português, Táticas, Técnicas e Procedimentos (TTP) efetivamente utilizados por adversários em incidentes reais (Strom *et al.*, 2018; ??). A diferença é importante: CVE e CWE descrevem o que pode ser explorado; o ATT&CK descreve como os atacantes agem ao longo das fases de uma intrusão, desde o reconhecimento inicial até a exfiltração de dados.

A matriz *Enterprise*, variante mais utilizada, organiza essas TTPs em 14 táticas e centenas de técnicas, cada uma identificada por um código — por exemplo, T1190 para *Exploit Public-Facing Application*. Em uma revisão sistemática, (Roy *et al.*, 2023) identificaram mais de 50 contribuições científicas que utilizam o ATT&CK como base para pesquisas em inteligência de ameaças, detecção de intrusão e resposta a incidentes, destacando-o como referência consolidada na academia e na indústria.

No agente proposto, o ATT&CK é utilizado para validar as *tags* de técnicas que o LLM inclui na regra *Sigma* gerada, visto que modelos de linguagem podem inventar identificadores inexistentes (por exemplo, *attack.exploitation* em vez do real *attack.initial-access*); a consulta à base oficial do ATT&CK permite remover ou sinalizar referências inválidas antes da entrega final da regra.

Cabe registrar que o MITRE mantém outras bases relacionadas, em particular o Common Attack Pattern Enumerations and Classifications (CAPEC), que cataloga padrões de ataque em um nível de abstração mais alto que o ATT&CK. Este catálogo não foi incorporado ao agente neste trabalho porque sua granularidade não corresponde ao formato das regras *Sigma*, que operam sobre técnicas concretas e mapeáveis a eventos de *log* — exatamente o nível em que o ATT&CK atua.

2.2.3 NVD e CVSS

O NVD é o repositório oficial do governo norte-americano para vulnerabilidades de segurança, mantido pelo NIST, e atua como camada de enriquecimento do CVE. A cada vulnerabilidade identificada, o NVD acrescenta metadados estruturados como produtos e versões afetados, mapeamentos para CWE e pontuações de severidade calculadas segundo o Sistema de Pontuação de Vulnerabilidades Comuns, do inglês *Common Vulnerability Scoring System* (CVSS) (National Institute of Standards and Technology, 2025; Booth; Rike; Witte, 2013). O

CVSS, por sua vez, é um sistema híbrido; produz uma pontuação numérica de 0,0 a 10,0 a partir de métricas como vetor de ataque, complexidade, privilégios requeridos e impacto sobre confidencialidade, integridade e disponibilidade, e mapeia essa pontuação em categorias qualitativas (*None, Low, Medium, High e Critical*), oferecendo simultaneamente critério quantitativo para ordenação e classificação intuitiva para consumo humano (Forum of Incident Response and Security Teams, 2019). No agente proposto, o NVD faz uma requisição para obter a descrição da vulnerabilidade, os CWEs associados e a pontuação CVSS — dados que alimentam tanto o conteúdo descritivo da regra quanto a calibração do campo *level* (que indica a criticidade da regra *Sigma*).

2.3 Regras *Sigma*

Sigma (*Generic Signature Format for SIEM Systems*, “Assinatura em formato genérico para sistemas SIEM” em tradução livre (SIGMAHQ, 2025b)) é o projeto de código aberto de um padrão genérico de descrição de *logs* de eventos, aplicável a qualquer tipo de *log*, com o intuito de compartilhar a lógica de detecção de ameaças ou anomalias entre diferentes SIEMs. Ao operar com dados de ameaças de múltiplas fontes, o *Sigma* capacita os caçadores de ameaças a correlacionar eventos que passariam despercebidos por abordagens convencionais. Dessa forma, se obtém conhecimentos estratégicos sobre dados confiáveis e antecipa-se tentativas de ataque em estágios iniciais desta cadeia de eventos (McGowan, 2025).

Os mantenedores da iniciativa consideram que excelentes análises podem ser encontradas na *internet*, incluindo IOCs e regras YARA² para detectar arquivos ou ferramentas maliciosos, por exemplo, mas que muitas vezes limitam-se a descrições específicas, funcionais para poucos *softwares*. Portanto, da necessidade de haver um método de descrição específico ou genérico de eventos de *logs*, com portabilidade para vários SIEMs, a fim de aprimorar os recursos de detecção para todos, surgiu o *Sigma* (SIGMAHQ, 2025b).

O projeto de código aberto foi criado em 2017 por Florian Roth e Thomas Patzke (SIGMAHQ, 2025a), inspirados em padrões já existentes nas áreas de análise de *malware* e monitoramento de rede. Como já dito, o padrão *Sigma* se diferencia por lidar com vários tipos de esquemas de *logs*, unindo padrões que já existem e se diferenciam entre si (McGowan, 2025).

O projeto carrega essa denominação pois se considera um ecossistema — envolve três componentes: o formato *Sigma*, a ferramenta *Sigma* e a coleção de regras *Sigma*. Este último diz respeito ao repositório no *Github*, que possui mais de 3 mil regras de detecção de diferentes tipos; por ser de código aberto, pode receber sugestões de qualquer pessoa disposta a contribuir (SIGMAHQ, 2025b). Já a ferramenta *Sigma*, refere-se a um conversor de regras disponível para *download* — o *Sigma Command Line Interface* (CLI). Com ele é possível converter as regras *Sigma* para as regras usadas no SIEM de interesse do usuário; *Elasticsearch*, *CrowdStrike*,

² Uma regra YARA é um arquivo de extensão “.yar” contido de expressões lógicas usadas para classificar e identificar amostras de *malware* (PICUS SECURITY, 2025).

IBM QRadar AQL e *Splunk* são algumas marcas registradas para as quais essa integração está disponível.

2.3.1 Sigma e a mitigação de ameaças

As regras *Sigma* formam um conjunto estruturado de princípios e métodos essenciais para o sucesso da investigação focada em determinar as ameaças que podem prejudicar um ou mais sistemas (*Threat Hunting*), oferecendo um padrão uniforme que orienta os profissionais de segurança na elaboração de regras criteriosas para detecção de ameaças ou anomalias. Segundo Dimitrov e Nikolov (2025), essas regras otimizam a identificação, avaliação e a resposta a incidentes, proporcionando decisões mais precisas e oportunas, além de uma base sólida para controlar vetores de ameaças, minimizar riscos e prevenir violações de dados. Dessa forma, as regras *Sigma* constituem o alicerce de uma estratégia de segurança proativa e resiliente, capaz de antecipar e mitigar possíveis ataques de forma sustentável.

A caça a ameaças é um processo defensivo proativo que visa descobrir e neutralizar invasores, mapeando vulnerabilidades e antecipando movimentos adversos por meio da análise contínua de comportamentos anômalos e incidentes em curso. Nesse cenário, as regras *Sigma*, desde sua padronização, em 2017, desempenham um papel fundamental ao oferecer uma linguagem comum para correlação de eventos em múltiplas plataformas SIEM, unificando dados de sistemas heterogêneos e acelerando a detecção de padrões suspeitos. Conforme demonstrado por Dimitrov e Nikolov (2025), essa unificação reduz significativamente a carga de trabalho manual dos analistas, permitindo identificar ameaças ainda na fase inicial de comprometimento, elevando a eficácia das operações de *Threat Hunting*, ou “caça a ameaças”.

No entanto, ainda que a aplicação das regras *Sigma* viabilize inúmeros cenários benquistas, vale ressaltar que, ainda assim, os SIEMs continuarão sendo heterogêneos, pois utilizam formatos e padrões de dados diferentes — situação inclusive de eventos de diferentes dispositivos. As regras *Sigma* surgem como uma forma de expressar a regra ou a ameaça e o que deve ser buscado, mas ainda precisa ser convertida para o SIEM de destino (através do CLI), que já possui suas próprias regras.

2.3.2 Estrutura

As regras *Sigma* são arquivos YAML contidos de informações que viabilizam a detecção de comportamento estranho ou malicioso quando da inspeção de arquivos de *logs* dentro do contexto de um SIEM (SIGMAHQ, 2025c).

A compreensão da estrutura de uma regra *Sigma* e como é usada para detecção faz-se importante em razão de que parte da metodologia envolverá selecionar as informações mais críticas das regras e avaliá-las.

As regras são divididas em três grupos principais de instruções (SIGMAHQ, 2025c):

- Detecção (*detection*): qual comportamento malicioso a regra está procurando;
- Fonte do log (*logsource*): quais tipos de logs a detecção deve procurar; e
- Metadados (*title*, *id*, *status*, etc.): detalhes da regra.

A Figura 1 é um exemplo de regra *Sigma* que está disponível na página oficial do projeto.

Figura 1 – Exemplo de regra *Sigma* (SIGMAHQ, 2025c)

```

1  # ./rules/cloud/okta/okta_user_account_locked_out.yml
2  title: Okta User Account Locked Out
3  id: 14701da0-4b0f-4ee6-9c95-2ffb4e73bb9a
4  status: test
5  description: Detects when a user account is locked out.
6  references:
7    - https://developer.okta.com/docs/reference/api/system-log/
8    - https://developer.okta.com/docs/reference/api/event-types/
9  author: Austin Songer @austinsonger
10 date: 2021-09-12
11 modified: 2022-10-09
12 tags:
13   - attack.impact
14 logsource:
15   product: okta
16   service: okta
17 detection:
18   selection:
19     displaymessage: Max sign in attempts exceeded
20     condition: selection
21 falsepositives:
22   - Unknown
23 level: medium

```

Fonte: Captura de tela da página *Sigma Rules* em SigmaHQ (2025c).

A seção de detecção é o coração da regra *Sigma*, pois especifica o alvo da regra no vasto volume de *logs*. Adiante neste trabalho este componente será mencionado como um dos principais para a análise da qualidade das regras geradas pelas IAs. Vale mencionar que, dentro da seção de detecção, existem instruções para a escrita, assim como qualquer linguagem de programação; no entanto, num primeiro momento, basta compreender o esqueleto principal. Sendo assim, a segunda peça mais importante da regra é a fonte dos *logs*, o campo *logsource*; nele se especifica qual o tipo de *log* deve ser canalizado pela regra — ao invés de varrer todos os *logs* do SIEM. Por fim, as demais informações, os metadados, servem para guiar o usuário sobre o assunto da regra (SIGMAHQ, 2025c).

Para fins de orientação quando da criação de uma nova regra, é disponibilizado um esqueleto da regra com instruções (Figura 2) no repositório *Sigma* (Roth, 2022).

Figura 2 – Exemplo de regra *Sigma* (Roth, 2022)

```

1 title: a short capitalised title with less than 50 characters
2 id: generate one here https://www.uuidgenerator.net/version4
3 status: experimental
4 description: A description of what your rule is meant to detect
5 references:
6   - A list of all references that can help a reader or analyst understand the meaning of a triggered rule
7 tags:
8   - attack.execution # example MITRE ATT&CK category
9   - attack.tl059      # example MITRE ATT&CK technique id
10  - car.2014-04-003    # example CAR id
11 author: Michael Haag, Florian Roth, Markus Neis # example, a list of authors
12 date: 2018/04/06 # Rule date
13 logsource: # important for the field mapping in predefined or your additional config files
14   category: process_creation # In this example we choose the category 'process_creation'
15   product: windows          # the respective product
16 detection:
17   selection:
18     FieldName: 'StringValue'
19     FieldName: IntegerValue
20     FieldName|modifier: 'Value'
21   condition: selection
22 fields:
23   - fields in the log source that are important to investigate further
24 falsepositives:
25   - describe possible false positive conditions to help the analysts in their investigation
26 level: one of five levels (informational, low, medium, high, critical)

```

Fonte: Captura de tela do código genérico na página *Rule Creation Guide* no repositório Sigma(Roth, 2022).

2.4 Processamento de Linguagem Natural

O Processamento de Linguagem Natural, do inglês *Natural Language Processing* (PLN) é a área da computação que busca fazer com que máquinas compreendam e produzam a linguagem humana. Segundo Liddy (2001), trata-se de uma abordagem teórica e tecnologicamente fundamentada para analisar textos, com o objetivo de alcançar um processamento de linguagem semelhante ao humano. Para isso, o PLN permite ao computador analisar um texto sistematicamente em sete níveis de representação: fonológico (padrões de entonação e fonemas), morfológico (estrutura das palavras), léxico (significado individual das palavras), sintático (estrutura gramatical das frases), semântico (significado literal das frases), discursivo (coesão e estrutura dos textos) e pragmático (contexto) liddy2001nlp, jurafsky2009speech. Ao longo do tempo, as técnicas evoluíram dos sistemas simbólicos baseados em regras manuais para abordagens estatísticas e, mais recentemente, conexionistas (inspiradas no funcionamento do cérebro humano), impulsionadas pelo aumento do poder computacional e pela disponibilidade de grandes coleções de texto (Liddy, 2001; Chowdhary, 2020).

Sobre essa base, o PLN passou a sustentar aplicações que se tornaram centrais na computação contemporânea. A Recuperação de Informação (RI) encontra, em uma grande coleção de documentos, aqueles mais relevantes a uma consulta, ordenando-os por similaridade semântica em vez de apenas casar palavras idênticas (Manning; Raghavan; Schütze, 2008). A Extração de Informação (EI) parte de texto livre e deste são retirados dados estruturados, como entidades, datas, relações, igualmente quando um anúncio de emprego em texto corrido é convertido em campos de uma tabela (Chowdhary, 2020). A Tradução Automática (TA) converte um texto entre idiomas preservando o significado original, mantendo uma correspondência pró-

xima entre origem e destino. Já a Sumarização condensa documentos longos em versões mais curtas que retêm o essencial, podendo ser extrativa (seleciona frases do original) ou abstrativa (reescreve em novas palavras) (Jurafsky; Martin, 2009).

Os Sistemas de Pergunta-Resposta (QA) ocupam um nível distinto: em vez de transformar diretamente o texto de entrada, interpretam uma pergunta em linguagem natural e produzem uma resposta direta, frequentemente combinando RI, EI e sumarização como etapas internas (Jurafsky; Martin, 2009). A diferença essencial é o tipo de saída entregue ao usuário — enquanto a RI devolve uma lista de documentos para que ele mesmo leia e conclua, o QA devolve a conclusão já formulada. Todas essas aplicações enfrentam dificuldades comuns relacionadas à integração de conhecimento de mundo e ao raciocínio de senso comum, necessários para resolver ambiguidades contextuais (Chowdhary, 2020). Esse cenário começou a mudar no final da década de 2010, com o surgimento dos modelos de linguagem baseados em redes neurais profundas, em especial os Modelos de Linguagem de Grande Escala, descritos na próxima seção, cuja escala de dados e parâmetros permitiu superar boa parte dessas limitações sem depender de regras escritas à mão (Hasanov *et al.*, 2024b).

Várias dessas aplicações reaparecem, de forma combinada, no agente desenvolvido neste trabalho. A recuperação de informação fundamenta a busca por regras Sigma semelhantes em uma base vetorial — núcleo da estratégia de geração aumentada por recuperação descrita na Seção 2.5. A extração de informação atua na identificação de indicadores técnicos, como identificadores CVE e *hashes*, a partir de texto livre. E a sumarização orienta a condensação do material coletado de fontes externas em uma descrição curta e factual da ameaça. Como se verá nas subseções seguintes, é justamente a possibilidade de unir essas tarefas em um único sistema, por meio de modelos de linguagem de grande escala, que torna essa arquitetura viável.

2.4.1 Modelos de Linguagem de Grande Escala (LLMs)

Os Modelos de Linguagem de Grande Escala (*Large Language Models*, LLMs) são sistemas de inteligência artificial treinados com bilhões ou trilhões de parâmetros, capazes de compreender e gerar textos em linguagem natural, ou seja, em formato de comunicação humano (Hasanov *et al.*, 2024b). Sua arquitetura é baseada em *Transformers*: redes neurais que empregam mecanismos de *self-attention* (autoatenção) para focar simultaneamente em diferentes partes do texto, capturando relações contextuais de longo alcance entre as palavras (Vaswani *et al.*, 2017). Esses modelos aprendem por meio de aprendizado auto-supervisionado (*self-supervised learning*), estratégia em que o próprio texto fornece o sinal de treinamento: dada uma sequência incompleta, o modelo tenta prever o elemento ausente, extraindo padrões linguísticos a partir de imensos *corpora*³ textuais sem rótulos explícitos (Hasanov *et al.*, 2024b).

³ *Corpora* é plural de *corpus* — termo em latim para "corpos" e, no contexto de PLN significa coleções extensas e estruturadas de textos autênticos.

O mecanismo operacional central de um LLM é a predição sequencial de *tokens* — as menores unidades de texto em que o modelo divide sua entrada. Dada uma sequência de entrada, o modelo estima a probabilidade do próximo token, e essa habilidade é refinada continuamente ao longo do pré-treinamento. Ao operar sobre janelas de contexto de milhares de *tokens*, o modelo mantém coerência semântica em textos longos e responde a instruções formuladas em linguagem natural (Brown *et al.*, 2020; Jurafsky; Martin, 2009).

Uma propriedade particularmente relevante para o presente trabalho é a capacidade dos LLMs de produzir saídas em formatos estruturados, como YAML, JSON ou código-fonte. É essa capacidade que permite ao modelo gerar regras *Sigma*, que são justamente "artefatos" estruturados em YAML, com campos formalmente definidos, e a arquitetura que viabilizou esse comportamento é o *Transformer*, descrita a seguir, enquanto as estratégias para orientar o modelo sem retreinamento são objeto da subseção **2.4.3**.

2.4.2 Arquitetura *Transformer*

A arquitetura *Transformer*, proposta por Vaswani *et al.* (2017) no artigo "*Attention is All You Need*", é o substrato técnico que viabilizou os modelos de linguagem contemporâneos. Antes dela, o processamento de texto era dominado por redes neurais recorrentes (RNN), que processavam o texto palavra por palavra, em sequência, o que dificultava o paralelismo computacional e limitava a captura de dependências entre elementos distantes.

O *Transformer* substitui a recorrência por um mecanismo denominado *self-attention* (autoatenção). Em vez de processar um *token* de cada vez, o modelo analisa todos os *tokens* da entrada simultaneamente e calcula, para cada um, o quanto os demais *tokens* da sequência são relevantes para sua representação. O resultado é uma representação contextualizada de cada *token*, que incorpora informação de toda a sequência (Vaswani *et al.*, 2017).

Essa formulação trouxe duas consequências práticas que explicam por que o *Transformer* se tornou a base dos LLMs. A primeira é o paralelismo: como todas as relações são calculadas simultaneamente, o treinamento torna-se massivamente paralelizável, viabilizando modelos com bilhões de parâmetros. A segunda é a captura de dependências de longo alcance: a relação entre dois tokens distantes é calculada com o mesmo custo da relação entre tokens adjacentes, eliminando o viés que penalizava informações distantes nas RNNs (Vaswani *et al.*, 2017). A arquitetura original combina um encoder, voltado à compreensão da entrada, e um decoder, voltado à geração da saída; os LLMs empregados neste trabalho derivam de sua porção decoder, especializada na geração de texto.

2.4.3 Engenharia de *Prompt*

Diferentemente dos modelos clássicos de aprendizado de máquina, que precisam ser retreinados para cada nova tarefa, os LLMs podem ser condicionados em tempo de inferência por meio de instruções em linguagem natural — os *prompts*. A formulação dessas instruções constitui o ramo da "engenharia de *prompt*", cujos paradigmas mais estabelecidos foram caracterizados por Brown *et al.* (2020).

O primeiro paradigma é o *zero-shot*, em que o modelo recebe apenas a descrição da tarefa e a entrada, sem nenhum exemplo de saída esperada. Instruir o modelo a "gerar uma regra *Sigma* para detectar a vulnerabilidade CVE-2021-44228" é uma chamada *zero-shot*: o modelo precisa inferir, a partir do que aprendeu no pré-treinamento, o formato esperado e os detalhes a incluir. O segundo paradigma é o *few-shot*, em que o *prompt* inclui um pequeno número de exemplos (tipicamente entre dois e dez) que ilustram a transformação desejada entre entrada e saída. Esses exemplos atuam como demonstração contextual: o modelo identifica o padrão a partir das amostras e o reproduz, sem que qualquer parâmetro seja atualizado (Brown *et al.*, 2020).

A diferença entre os dois tem implicações práticas. O *zero-shot* depende inteiramente do conhecimento incorporado no pré-treinamento, sendo mais sensível a domínios pouco representados nos dados de origem; o *few-shot* fornece especificação adicional sem custo de treinamento, mas consome parte do orçamento de *tokens* da janela de contexto. Brown *et al.* (2020) demonstraram que o ganho do *few-shot* sobre o *zero-shot* é mais expressivo em modelos de maior escala.

Estratégias mais elaboradas combinam essas modalidades com instruções estruturais. O *Chain-of-Thought* (CoT), proposto por Wei *et al.* (2022), instrui o modelo a externalizar seu raciocínio em passos intermediários antes de produzir a resposta final, o que demonstrou ganhos significativos em tarefas que exigem composição lógica. Em vez de gerar diretamente a saída, o modelo é induzido a "pensar em voz alta", decompondo o problema e justificando decisões parciais, o que tende a reduzir erros em tarefas estruturadas.

No presente trabalho, esses paradigmas constituem objeto direto de investigação experimental. O Estágio 2 da metodologia emprega geração *zero-shot* como linha de base; o Estágio 3 aplica engenharia de *prompt* estruturada, com instruções refinadas, seções nomeadas e restrições de formato; e o Estágio 4 incorpora recuperação aumentada de exemplos via RAG, configurando uma forma de *few-shot* contextualizada, combinada a uma etapa explícita de CoT, em que o agente é instruído a raciocinar sobre a fonte de *log*, os campos relevantes, os modificadores apropriados e a técnica ATT&CK correspondente antes de emitir a regra *Sigma* final. A comparação controlada entre essas estratégias permite quantificar o ganho marginal de cada camada de complexidade, conforme detalhado no Capítulo 3.

2.5 Geração Aumentada por Recuperação (RAG)

2.6 Agentes Autônomos

3 TRABALHOS RELACIONADOS

3.1 *LLMCloudHunter*

O *LLMCloudHunter* é uma proposta de Schwartz *et al.* (2025), um *framework* que utiliza o modelo GPT-4o para automatizar a geração de regras *Sigma* a partir de relatórios de código aberto de inteligência de ameaças, voltados a ambientes em nuvem (*Open-Source Cyber Threat Intelligence* (OSCTI)). Os autores identificaram três limitações em trabalhos anteriores que motivaram a pesquisa: a ausência de saídas acionáveis, prontas para serem aplicadas em sistemas de segurança; o desperdício de informações visuais presentes em relatórios de ameaças, como capturas de tela e diagramas de fluxo de ataque; e o foco em ambientes *on-premises* (infraestrutura local), em detrimento dos ambientes em nuvem. Para superar essas lacunas, o *framework* utiliza uma abordagem multimodal, ou seja, com módulos que processam texto e imagens de forma conjunta.

A metodologia se organiza em três fases. Na fase de pré-processamento, a ferramenta *BeautifulSoup* é empregada no *web scraping* do código HTML do relatório, onde se descartam elementos irrelevantes – como barras laterais e propagandas; o conteúdo restante é convertido em um formato de texto unificado, no qual cabeçalhos, listas e tabelas têm sua estrutura hierárquica preservada. Em seguida, o componente *Image Analyzer* interpreta cada imagem do relatório por meio das capacidades visuais nativas do GPT-4o, transcreve seu conteúdo para texto e o reintegra à posição original no documento.

Na segunda fase o processamento é em nível de parágrafo, onde acontece a extração de entidades (pedaço de informação relevante) e a geração inicial dos candidatos à regra final. O componente *API Call Extractor* atua em duas etapas: primeiro extrai chamadas de API explicitamente citadas no texto e, em seguida, infere chamadas mencionadas de forma implícita. A inferência é necessária porque relatórios de ameaças frequentemente descrevem operações de alto nível que não correspondem diretamente a eventos registrados nos *logs*. Conforme ilustram os próprios autores, "descrições operacionais, como a realização de uma ação de sincronização (*sync*) em um *bucket* S3, devem ser mapeadas para as chamadas de API *ListBuckets* e *GetObject*" (Schwartz *et al.*, 2025, p. 1925). Para mitigar alucinações e omissões do modelo, esse componente incorpora um mecanismo de votação por maioria, executando a extração múltiplas vezes e mantendo apenas as ocorrências que ultrapassam um valor de referência pré-definido. Depois, o *TTP Extractor* mapeia cada chamada de API às respectivas táticas, técnicas e subtécnicas do MITRE ATT&CK, e o *Rule Generator* sintetiza essas informações na estrutura sintática das regras *Sigma*. Por fim, o *Rule Validator* higieniza os candidatos gerados, remove campos excessivamente específicos e ajusta a sintaxe ao padrão *Sigma*.

Na terceira fase, de processamento em nível de OSCTI, o escopo de refinamento é global – o *framework* olha todos os candidatos juntos para tomar decisões. O *Rule Optimizer* executa duas operações lógicas: unificação, que mescla campos de seleção que compartilham

os mesmos critérios de detecção; e separação, que divide campos com lógicas divergentes incorretamente agrupados. Em seguida, o *Set Refiner* elimina redundâncias decorrentes do processamento paralelo dos parágrafos, selecionando, para cada chamada de API repetida em múltiplos candidatos, a regra mais adequada e descartando ou ajustando as demais. Paralelamente, o *IoC Extractor* identifica endereços IP e *User Agents* citados no relatório, e em seguida o *IoC Enhancer* os integra ao conjunto final como campos de seleção adicionais, conectados à lógica original por meio de operadores condicionais, sem comprometer a estrutura de detecção previamente estabelecida.

Os resultados de Schwartz *et al.* (2025) demonstram que, na identificação das chamadas de API (sequência de comandos) atribuídas ao agente da ameaça nos relatórios analisados, o *framework* acertou 83% das extrações e recuperou 99% das ocorrências presentes nos relatórios. Para os indicadores de comprometimento (IOC), os índices foram de 99% e 97%, respectivamente. A avaliação foi conduzida sobre uma base de vinte relatórios reais de ameaças em nuvem, previamente anotados de forma manual pela equipe de pesquisa. Além disso, 99,18% das regras *Sigma* geradas pelo *framework* foram compiladas com sucesso e convertidas em consultas para a plataforma *Splunk*, o que indica que a saída produzida atende ao padrão esperado. O estudo dos autores confirmou ainda a importância do processamento das imagens contidas nos relatórios, evidenciando que a leitura conjunta de texto e elementos visuais é uma estratégia adequada para lidar com a diversidade de formatos encontrados em relatórios de ameaças em nuvem (Schwartz *et al.*, 2025, seção 4.3).

3.2 RulePilot

O *RulePilot*, derivado do estudo de Wang *et al.* (2026), trata-se de um agente baseado em LLM para geração e conversão automatizada de regras de detecção em sistemas SIEMs proprietários, com foco principal em regras escritas na linguagem do *Splunk* (SPL), com suporte adicional à linguagem do *Microsoft Sentinel* (a *Kusto Query Language* (KQL)). Nas motivações do trabalho, os autores justificam o foco em regras específicas de cada SIEM sob a argumentação de que formatos genéricos como o *Sigma*, apesar de sua portabilidade entre diferentes plataformas e fornecedores, dependem majoritariamente de correspondência de padrões por expressões regulares e não suportam consultas complexas como agregações estatísticas, junções multi-fonte ou cálculos em janelas temporais, tendo, portanto, limitações na eficácia de detecção de ataques sofisticados que envolvem sequências de eventos. Wang *et al.* (2026) citam como referência de trabalhos anteriores de geração automática de regras o próprio *LLM-CloudHunter* (Schwartz *et al.*, 2024) – citado na seção anterior do presente trabalho. Os autores sustentam que *frameworks* como o citado contém tais limitações por restringirem-se ao formato *Sigma*. Portanto, a motivação central do *RulePilot* é suprir a lacuna entre descrições de alto nível em linguagem natural e as regras complexas, específicas de cada fornecedor, requeridas por SIEMs em produção.

A arquitetura do *framework RulePilot* é organizada em três componentes principais. O primeiro é o módulo de raciocínio em cadeia (*Chain-of-Thought* (CoT)), no qual a descrição de entrada é decomposta em seis etapas estruturadas: interpretação dos objetivos de segurança, identificação das fontes de *logs*, definição de filtros iniciais, extração de campos relevantes, agregação de dados e otimização da regra. O segundo é a representação intermediária (*Intermediate Representation* (IR)), uma abstração estruturada que separa a lógica semântica da sintaxe específica do SIEM-alvo: `<KEYWORD> | PARAMS {k_i=v_i} | MODULES {m_j}`; *KEYWORD* designa a função da etapa (como *FILTER*, *EXTRACT*, *AGGREGATE* e *JOIN*); *PARAMS* define as configurações obrigatórias e *MODULES* descreve a intenção analítica. Essa separação permite que o LLM se concentre no raciocínio semântico antes de se comprometer com a sintaxe final. O terceiro componente é o mecanismo de reflexão e otimização iterativa, que avalia a regra gerada em três dimensões – consistência lógica, correção sintática e viabilidade de execução, e dispara dois tipos de refinamento: regeneração de etapas da cadeia de raciocínio e a integração com a API do *Splunk* real para validar com execução de *logs* verdadeiros. Para a funcionalidade de conversão entre linguagens SIEM, o *RulePilot* incorpora ainda um mecanismo de RAG alimentado pela documentação oficial de migração da *Microsoft*, que mapeia comandos do SPL para seus equivalentes em KQL por meio de busca vetorial combinada à re-ranqueamento lexical. O *RulePilot* foi avaliado utilizando três LLMs distintos como motor – GPT-4o, DeepSeek-V3 e LLaMA-3, sobre 1.699 regras oficiais do repositório *Splunk Security Content* e 229.968 *logs* coletados de 61 testes atômicos do ATT&CK.

Os resultados de Wang *et al.* (2026) afirmam que o *RulePilot* supera os LLMs autônomos (sem o *framework*) em até 107,4% na similaridade textual com as regras de referência, medida pelas métricas BLEU, ROUGE-1, ROUGE-L e METEOR.¹ Na avaliação de execução real sobre o *Splunk*, o *RulePilot* atingiu precisão de 100% em diversas táticas do ATT&CK, incluindo reconhecimento, acesso inicial e coleta, com desempenho superior ao -4o autônomo. Tais ganhos, contudo, acompanham certo custo computacional: o *RulePilot* consome cerca de dez vezes mais *tokens* de *prompt* e aproximadamente seis vezes mais tempo de geração que o LLM cru, em razão de seu raciocínio em múltiplas etapas e dos ciclos de refinamento iterativo. O estudo de ablação serve para entender o papel de cada parte do *RulePilot*. A ideia é simples: tira-se uma parte de cada vez para ver o quanto o resultado piora, assim se descobre para quê serve cada uma. O *RulePilot* tem duas partes principais: a primeira é a representação intermediária (IR), que ajuda a montar a regra com a sintaxe certa; quando ela é removida, o produto é uma regra com mais erros de escrita e de difícil leitura – a nota média cai de 0,55 para 0,44. A segunda parte é o raciocínio em cadeia com revisão (*Chain-of-Thought + Reflection* (CoT-R)) – ela ajuda a regra a ter sentido do começo ao fim. Quando se remove este bloco de raciocínio

¹ BLEU (Papineni *et al.*, 2002), ROUGE (Lin, 2004) e METEOR (Banerjee; Lavie, 2005) são métricas automáticas que medem a similaridade entre um texto gerado e uma referência pela sobreposição de n-gramas. BLEU prioriza precisão e ROUGE prioriza revocação; METEOR acrescenta sinônimos, radicais e ordem das palavras. Valores mais altos indicam maior alinhamento textual.

em cadeia e reflexão, o que mais piora é a consistência lógica e a cobertura de condições. A nota média cai para 0,41. Quando as duas partes são removidas juntas, o resultado é o pior de todos: a nota cai para 0,35. Ou seja, as duas se completam – cada uma cuida de um problema diferente. No fim, os autores testaram se o *RulePilot* consegue traduzir regras de uma plataforma para outra. Em regras simples, ele acertou tudo (nota 1,000). Em regras mais complicadas, que juntam dados de várias fontes, acertou quase tudo (nota 0,919). Isso mostra que dá para migrar regras de um sistema para outro sem grandes perdas. Por fim, os autores avaliaram a conversão entre linguagens SIEM, e o *RulePilot* atingiu nota 1,000 em F1, nas regras de agregação e de listagem. Já nas regras com junções de várias fontes, que são mais complexas, o F1 foi de 0,919. Esse resultado mostra que a migração entre plataformas SIEM diferentes é tecnicamente viável.

3.3 GRIDAI

O estudo de Li *et al.* (2025) (ainda apenas disponível como pré-print) propõe o GRIDAI, um *framework* de ponta a ponta para geração e reparo automatizado de regras de detecção de intrusão em sistemas baseados em assinaturas, como o Suricata e Snort. São citados alguns trabalhos anteriores que buscaram automatizar a criação dessas regras, como AutoCombo (Du; Hu; Hewlett, 2021) e Syrius (Alcantara *et al.*, 2021); entretanto, Li *et al.* (2025) argumentam que limitação central deles é que todos geram uma regra nova para cada amostra de ataque recebida e precisam de tráfego de rede normal como referência, sem nunca verificar se uma regra já existente poderia simplesmente ser atualizada. Com o tempo, isso gera um acúmulo de regras redundantes e um volume crescente de alertas duplicados, dificultando a manutenção do sistema. O GRIDAI propõe uma lógica diferente: antes de agir, ele avalia a natureza de cada amostra. Se o ataque for inédito, cria uma nova regra; se for uma variação de algo já conhecido, atualiza a regra existente para que ela passe a cobrir também esse novo caso. A arquitetura é composta por quatro módulos especializados coordenados por uma lógica central. O primeiro módulo (*Relation-Assess Agent*) funciona em dois passos: primeiro testa a amostra diretamente no motor de detecção e, somente se nenhum alerta for gerado, consulta o modelo de linguagem para decidir o que fazer. O segundo (*New-Rule-Generation Agent*) cria regras novas analisando o conteúdo do ataque em etapas, gerando e testando múltiplos candidatos. Já o terceiro (*Existing-Rule-Repair Agent*), ajusta regras existentes e garante que a versão modificada continue funcionando tanto para o ataque original quanto para a nova variante. E o quarto (*Memory-Update Agent*) registra todas as alterações. Para lidar com erros gerados pelos modelos de linguagem, cada regra produzida é validada em tempo real no motor de detecção; se as falhas ultrapassarem um limite pré-definido, o sistema desfaz a operação e tenta novamente.

Os experimentos foram realizados em dois conjuntos de dados – um sintético (com sete tipos de ataques) e um real (com quarenta e três tipos), além de usar o GPT-4.1 como modelo principal. O sistema também foi testado com outros quatro modelos de linguagem (GPT-4o,

GPT-3.5-turbo, GLM-4-Plus e GLM-4-Flash), demonstrando que o GRIDAI tem um bom funcionamento na utilização de modelos distintos. Quando na tarefa de identificar corretamente o que fazer com cada amostra, o sistema acertou 100% das decisões no conjunto sintético e 96,9% dos casos de reparo no conjunto real; a acurácia para geração de novas regras foi de 60,5%, em virtude de algumas regras do conjunto de referência compartilharem características muito similares entre si. Frente aos três trabalhos tomados como base de comparação por Li *et al.* (2025), o GRIDAI detectou 100% dos ataques no conjunto sintético e 95,4% no conjunto real, sem gerar nenhum falso alarme. O experimento de ablação, em que os autores removeram individualmente cada um dos principais mecanismos do sistema para medir o impacto de cada componente no desempenho final, mostrou que sem o mecanismo de reparo, a taxa de alertas duplicados sobe de 12,7% para 66,8%, confirmando que reaproveitar e atualizar regras existentes é muito mais eficiente do que criar novas regras a cada variante detectada.

3.4 Aprendizado de máquina aplicado a SIEMs e análise de logs

A aplicação de aprendizado de máquina para análise automatizada de logs e a identificação de ameaças foi explorada de forma convergente por Shah *et al.* (2022) e Nurusheva *et al.* (2024). Ambos trabalhos partem da constatação de que o volume e a heterogeneidade dos logs gerados por sistemas modernos inviabilizam a inspeção manual, e as abordagens tradicionais baseadas somente em regras estáticas falham na identificação de ataques novos ou ofuscados². Para superar essa limitação, os dois estudos recorrem a técnicas de ML, embora com metodologias distintas – empírico-quantitativa com foco em agrupamento no primeiro trabalho e com foco arquitetural e demonstrativo no segundo.

Shah *et al.* (2022) propõe um modelo que enfrenta o desafio da ausência de dados rotulados, problema recorrente em cenários reais de monitoramento. A metodologia inicia com pré-processamento dos registros, mantendo apenas o conteúdo textual das mensagens. Em seguida, os autores aplicam a vetorização Frequência de Termo–Frequência Inversa de Documento, do inglês *Term Frequency–Inverse Document Frequency* (TF-IDF) e executam o algoritmo de *clustering* não-supervisionado *K-means*. A partir dos agrupamentos obtidos, os pesquisadores rotularam os registros e geraram um conjunto de dados para treinar um classificador supervisionado *Naive Bayes*. Avaliado sobre o *dataset BlueGene/L* (BGL), o modelo obtido alcançou acurácia média de 98% na predição de eventos anômalos, demonstrando viabilidade para ambientes que exigem triagem rápida.

Já Nurusheva *et al.* (2024), propõem a integração direta de algoritmos com ML supervisionado a um sistema SIEM. A validação foi conduzida em uma arquitetura construída sobre a pilha ELK, com cenários de ciberataques simulados na aplicação propositalmente vulnerável chamada *Damn Vulnerable Web Application*, do inglês *Damn Vulnerable Web Application*

² Ofuscação é a técnica de mascarar a aparência de um código, comando ou tráfego de rede para torná-lo incompreensível aos sistemas de detecção e analistas, sem alterar a função executável original.

(DVWA). Os autores treinaram modelos clássicos – *Support Vector Machine*, *Random Forest*, *Decision Tree* e *AdaBoost*, além de uma Rede Neural Convolucional, do inglês *Convolutional Neural Network* (CNN) adaptada à classificação de eventos de segurança, utilizando amostras de ataques baseadas no *framework*. Como resultado, relataram ganhos operacionais: a correlação de eventos reduziu falsos positivos e a automação da resposta diminuiu a carga sobre os analistas de segurança. Diferentemente do estudo anterior, o trabalho de Nurusheva *et al.* (2024) possui caráter predominantemente descritivo, com foco na arquitetura operacional, em detrimento de uma avaliação baseada em métricas comparativas rigorosas.

Ao analisar conjuntamente as duas pesquisas, vê-se um dilema prático na modernização e evolução da segurança cibernética: o equilíbrio entre o refinamento algorítmico isolado e a aplicabilidade sistêmica. Enquanto a abordagem de Shah *et al.* (2022) comprova que é possível contornar a escassez de dados rotulados por meio de *pipelines* híbridos de aprendizado de máquina, alcançando alta acurácia em ambientes controlados, a proposta de Nurusheva *et al.* (2024) evidencia que o verdadeiro desafio está na orquestração contínua desses modelos dentro de ecossistemas corporativos reais. Portanto, a consolidação do uso de aprendizado de máquina na triagem de eventos de segurança permanece condicionada à superação dessa fronteira: trabalhos como o de Shah *et al.* (2022) demonstram a viabilidade técnica do componente preditivo, enquanto Nurusheva *et al.* (2024) ilustram a integração operacional desses componentes em SIEMs – mas raramente os dois aspectos são tratados de forma articulada em um único estudo.

3.5 Posicionamento do Trabalho Proposto

O conjunto de trabalhos analisados nas seções anteriores expõe duas vertentes complementares na pesquisa em automação da detecção de ameaças. A primeira vertente se concentra em técnicas de aprendizado de máquina aplicadas à triagem e classificação de *logs* de segurança, sem produzir artefatos sintáticos diretamente acionáveis em sistemas SIEM. Nesta vertente encontram-se Shah *et al.* (2022) e Nurusheva *et al.* (2024), que combinam métodos supervisionados e não supervisionados para reduzir a carga de alertas em ambientes corporativos – estabelecendo, ainda que indiretamente, um terreno metodológico sobre o qual as abordagens mais recentes puderam se apoiar. A segunda vertente se concentra na geração automatizada das próprias regras de detecção, valendo-se das capacidades generativas dos LLMs, e representa uma evolução natural da primeira: uma vez consolidada a aplicabilidade de aprendizado de máquina e inteligência artificial na análise de eventos de segurança, abre-se caminho para que essas mesmas técnicas atuem não apenas na classificação dos eventos, mas também na produção dos próprios mecanismos de detecção. Nesta segunda vertente está o GRIDAI ((Li *et al.*, 2025)), voltado para regras de sistemas de detecção de intrusão de rede; o *RulePilot* ((Wang *et al.*, 2026)), voltado para regras nativas das linguagens SPL e KQL; e o *LLMCloudHunter* ((Schwartz *et al.*, 2025)), voltado para regras *Sigma* em ambientes de nuvem.

O presente trabalho melhor se insere na segunda vertente, compartilhando com Schwartz *et al.* (2025) o objetivo de geração de regras *Sigma*.

A análise comparativa dos estudos revela três observações fundamentais para o posicionamento da presente pesquisa. Num primeiro momento, dentre os cinco trabalhos analisados, apenas o *LLMCloudHunter* de Schwartz *et al.* (2025) compartilha exatamente o mesmo formato-alvo de saída (regras *Sigma*). No entanto, mesmo neste caso, há diferenças relevantes de escopo: o *LLMCloudHunter* opera sobre relatórios OSCTI como entrada e produz regras específicas à detecção em ambientes de nuvem AWS, enquanto o presente trabalho opera a partir de registros CVE estruturados e notícias de vulnerabilidade, visando a produção de regras *Sigma* não restritas a um provedor de nuvem específico. Em segundo, os trabalhos voltados a outras linguagens de regra – Li *et al.* (2025) e Wang *et al.* (2026), não competem diretamente no domínio *Sigma*, mas oferecem precedentes arquiteturais relevantes para a estruturação do agente proposto neste trabalho. O uso de validação iterativa em motor de detecção real ((Li *et al.*, 2025)) e o uso de raciocínio em cadeia com reflexão ((Wang *et al.*, 2026)) são padrões metodológicos que se aproximam da arquitetura LangGraph empregada nesta pesquisa, ainda que a aplicação tenha contextos sintáticos distintos. Em terceiro, os trabalhos da primeira vertente – Shah *et al.* (2022) e Nurusheva *et al.* (2024), embora não produzam regras como saída, são fundamentais para contextualizar a presente pesquisa: comprovam o quão útil é o uso de técnicas de aprendizado de máquina e inteligência artificial na detecção de ameaças em sistemas SIEM, fundamentando a premissa de que tais técnicas, hoje já consolidadas, estendem-se para tarefas sofisticadas como a geração automatizada de regras de detecção.

Há ainda uma divergência teórica interessante entre os trabalhos analisados, que merece saliência. Wang *et al.* (2026) argumentam que o formato *Sigma* é insuficiente para a detecção de ataques sofisticados que envolvem sequências de eventos, por restringir-se sobretudo à correspondência de padrões por expressões regulares, criticando trabalhos que adotam *Sigma* como saída – citando diretamente o *LLMCloudHunter*. O presente trabalho optou por manter o foco no formato *Sigma* em decorrência de sua portabilidade entre múltiplos desenvolvedores de SIEMs, da existência de um repositório público bem consolidado de regras (o SigmaHQ) que serve como base para o mecanismo de RAG, e da posição do formato como padrão aberto mantido pela comunidade de *detection engineering*. Essa escolha posiciona o trabalho aqui proposto em alinhamento com Schwartz *et al.* (2025), mas com clara consciência das limitações expressivas do formato apontadas por Wang *et al.* (2026). Vale ressaltar igualmente que a própria arquitetura proposta para o agente desenvolvido neste trabalho – fundamentada no uso de RAG sobre o repositório SigmaHQ, pode apresentar limitações na tarefa específica de geração de regras *Sigma*, conforme indícios observados ao longo do desenvolvimento. Portanto, investigação empírica conduzida neste trabalho de conclusão de curso pretende vislumbrar o quanto essa estratégia se sustenta frente às alternativas avaliadas, contribuindo para o esclarecimento dessa questão metodológica.

Ante o exposto, dessa análise emerge o recorte ocupado pelo presente trabalho. Enquanto o *LLMCloudHunter* demonstra a viabilidade técnica da geração de regras *Sigma* via LLM em um domínio específico, e enquanto GRIDAI e *RulePilot* exploram arquiteturas agênticas em outras linguagens de regra, permanece em aberto a questão metodológica de quanto a complexidade arquitetural – desde a invocação direta de uma LLM em formato *zero-shot* até a orquestração de um agente autônomo com RAG e validação iterativa, efetivamente contribui para a qualidade das regras *Sigma* geradas a partir de fontes públicas de inteligência sobre ameaças. O estudo aqui exposto investiga essa questão por meio da comparação controlada entre três estratégias incrementais — *zero-shot*, engenharia de *prompt* e agente autônomo com RAG – submetidas a teste estatístico de hipóteses. Diferentemente dos cinco trabalhos analisados, que tipicamente comparam uma única abordagem proposta contra LLMs autônomos ou métodos baseados em mineração de dados, o presente trabalho se propõe a quantificar empiricamente o ganho marginal proporcionado por cada camada de complexidade arquitetural na tarefa de geração de regras *Sigma*, fornecendo subsídios para decisões de *design* em trabalhos futuros na linha de pesquisa.

4 MATERIAIS E MÉTODOS

Este capítulo descreve os materiais e os métodos que serão utilizados para conduzir esta pesquisa. Apresentar-se-ão, o conjunto de ferramentas e bibliotecas empregadas no desenvolvimento; a base de dados de regras *Sigma* utilizada como referência, juntamente com seu particionamento em base de conhecimento e base de teste; as três estratégias de geração de regras que constituem o objeto de comparação deste trabalho — geração *zero-shot*, engenharia de *prompt* e agente autônomo com RAG; as métricas de avaliação adotadas; e, por fim, a formulação das hipóteses e o procedimento de análise estatística.

4.1 Materiais

Quadro 1 – Modelos de linguagem utilizados no trabalho.

Ferramenta	Versão	Disponível em	Finalidade
ChatGPT	—	https://chat.openai.com/	LLM comercial utilizada nas fases de geração <i>zero-shot</i> e engenharia de <i>prompt</i> .
Gemini	—	https://gemini.google.com/	LLM comercial utilizada nas fases de geração <i>zero-shot</i> e engenharia de <i>prompt</i> .
Claude	—	https://claude.ai/	LLM comercial utilizada nas fases de geração <i>zero-shot</i> e engenharia de <i>prompt</i> .
Microsoft Copilot	—	https://copilot.microsoft.com/	LLM comercial utilizada nas fases de geração <i>zero-shot</i> e engenharia de <i>prompt</i> .
DeepSeek	—	https://chat.deepseek.com/	LLM comercial utilizada nas fases de geração <i>zero-shot</i> e engenharia de <i>prompt</i> .
Llama 3.1 Instruct	8B	https://ollama.com/library/llama3.1	LLM de código aberto utilizada como geradora de regras <i>Sigma</i> nas três estratégias avaliadas e como modelo principal do agente autônomo.
Qwen 2.5	1.5B	https://ollama.com/library/qwen2.5	LLM alternativa, de menor consumo de memória, utilizada na modelagem do agente por motivos de restrições de <i>hardware</i> .
bge-small-en-v1.5	—	https://huggingface.co/BAAI/bge-small-en-v1.5	Modelo de <i>embeddings</i> (384 dimensões) utilizado para vetorização das regras <i>Sigma</i> e para o cálculo da similaridade semântica.
ms-marco-MiniLM-L-12-v2	—	https://huggingface.co/ms-marco-MiniLM-L-12-v2	Modelo <i>cross-encoder</i> utilizado na etapa de <i>re-ranking</i> , refinando o ordenamento dos candidatos recuperados por similaridade.

Fonte: Autoria própria (2026).

Quadro 2 – *Frameworks, banco vetorial e ambiente de desenvolvimento.*

Ferramenta	Versão	Disponível em	Finalidade
<i>langchain-core</i>	1.4.1	https://pypi.org/project/langchain-core/	Núcleo do <i>framework</i> LangChain; fornece as abstrações de mensagens utilizadas pelo gerador.
<i>langchain-community</i>	0.4.2	https://pypi.org/project/langchain-community/	Integrações comunitárias; fornece os <i>loaders</i> de documentos utilizados na construção do banco vetorial.
<i>langchain-ollama</i>	1.1.0	https://pypi.org/project/langchain-ollama/	Integração entre o LangChain e o servidor Ollama; fornece a classe <i>ChatOllama</i> .
<i>langchain-chroma</i>	1.1.0	https://pypi.org/project/langchain-chroma/	Integração entre o LangChain e o ChromaDB, utilizada pelo mecanismo de RAG.
<i>langchain-huggingface</i>	1.2.2	https://pypi.org/project/langchain-huggingface/	Integração entre o LangChain e a Hugging Face; carrega o modelo de <i>embeddings</i> .
<i>langgraph</i>	1.2.4	https://langchain-ai.github.io/langgraph/	Biblioteca utilizada para modelar o agente como uma máquina de estados, com nós, arestas condicionais e estado compartilhado.
<i>sentence-transformers</i>	5.5.1	https://sbert.net/	Biblioteca que provê as classes utilizadas para carregar os modelos de <i>embeddings</i> e o <i>cross-encoder</i> .
Ollama	—	https://ollama.com/	Servidor local para inferência de LLMs de código aberto, permitindo executar o modelo gerador em <i>hardware</i> próprio.
<i>chromadb</i>	1.5.9	https://www.trychroma.com/	Banco de dados vetorial utilizado para persistir os <i>embeddings</i> das regras <i>Sigma</i> e suportar a recuperação por similaridade.
Visual Studio Code	—	https://code.visualstudio.com/	Ambiente de desenvolvimento integrado utilizado para a implementação.

Fonte: Autoria própria (2026).

Conforme apresentado no Quadro 4, para o desenvolvimento do trabalho se utilizou dois equipamentos. O *laptop*, com *Graphics Processing Unit* (GPU) integrada e recursos de memória mais limitados, foi utilizado no início do trabalho em virtude de problemas técnicos com o *desktop*. Graças a este incidente, a fase de prototipação do agente se deu com o modelo *Qwen 2.5 1.5B*, que demanda menos computacionalmente e permitiu iterar rapidamente sobre a arquitetura. O *desktop*, equipado com GPU NVIDIA RTX 3060 e 12 GB de VRAM dedicada, foi empregado na execução dos experimentos finais e na geração das regras *Sigma* com o modelo *Llama 3.1 8B*.

Os materiais apresentados no 1, atendem a duas finalidades distintas. As LLMs comerciais *ChatGPT*, *Gemini*, *Claude*, *Microsoft Copilot* e *DeepSeek* foram utilizadas exclusivamente

Quadro 3 – Bibliotecas de validação, coleta de dados e análise estatística.

Ferramenta	Versão	Disponível em	Finalidade
Python	3.12.3	https://www.python.org/	Linguagem utilizada para implementar o agente, os <i>scripts</i> de geração de <i>datasets</i> e os procedimentos de avaliação.
<i>pySigma</i>	1.3.3	https://github.com/SigmaHQ/pySigma	Biblioteca oficial mantida pela SigmaHQ, utilizada na validação semântica das regras geradas.
<i>PyYAML</i>	6.0.3	https://pyyaml.org/	Biblioteca para análise sintática (<i>parsing</i>) de YAML, utilizada na validação de primeiro nível das regras geradas.
<i>requests</i>	2.34.2	https://requests.readthedocs.io/	Biblioteca utilizada para requisições HTTP a fontes públicas de inteligência sobre ameaças.
<i>beautifulsoup4</i>	4.15.0	https://www.crummy.com/software/BeautifulSoup/	Biblioteca utilizada para <i>parsing</i> de HTML das páginas de referência recuperadas pelo agente.
<i>ddgs</i>	9.14.4	https://pypi.org/project/ddgs/	Biblioteca utilizada para busca <i>web</i> complementar quando as referências diretas são insuficientes.
<i>scipy</i>	1.17.1	https://scipy.org/	Biblioteca utilizada nos testes estatísticos não paramétricos (Wilcoxon, Friedman, McNemar).
<i>statsmodels</i>	—	https://www.statsmodels.org/	Biblioteca utilizada para o teste Q de Cochran e a ANOVA de medidas repetidas.
<i>scikit-learn</i>	1.9.0	https://scikit-learn.org/	Utilizada para o cálculo da similaridade do cosseno entre <i>embeddings</i> de regras.

Fonte: Autoria própria (2026).

Quadro 4 – Hardware utilizado no desenvolvimento do trabalho.

Componente	Desktop (principal)	Laptop (prototipação)
Modelo	ASUS TUF H310M-PLUS GAMING/BR	Lenovo IdeaPad 3 15ALC6
Processador	Intel Core i7-9700K @ 3.60 GHz (8 núcleos)	AMD Ryzen 7 5700U @ 1.80 GHz (8 núcleos / 16 <i>threads</i>)
Memória RAM	16 GB DDR4 2133 MHz (2×8 GB)	12 GB DDR4 3200 MHz (8 GB + 4 GB)
GPU	NVIDIA GeForce RTX 3060 (12 GB GDDR6 dedicados)	AMD Radeon Lucienne (integrada, sem VRAM dedicada)
Armazenamento	SSD Kingston SA400S3 480 GB (SATA)	SSD NVMe SSSTC 512 GB
Sistema operacional	Ubuntu Linux	Ubuntu Linux

Fonte: Autoria própria (2026).

nas duas primeiras fases do experimento (geração *zero-shot* e engenharia de *prompt*), em que cada modelo recebe o mesmo conjunto de entradas e gera regras *Sigma* de forma independente, permitindo a comparação entre diferentes provedores. Já os modelos de código aberto (*Llama 3.1* e *Qwen 2.5*), juntamente com os *frameworks* *LangChain*, *LangGraph* e *ChromaDB*, constituem a infraestrutura do agente autônomo desenvolvido na terceira fase, sendo responsáveis pela geração, recuperação de contexto via RAG, e validação iterativa das regras. Os modelos de *embeddings* e *re-ranking* (*bge-small-en-v1.5* e *ms-marco-MiniLM-L-12-v2*) são componentes internos do agente, utilizados na etapa de recuperação e não participam diretamente da geração de texto.

4.1.1 Base de Dados

Tabela 1 – Composição das bases de dados utilizadas no trabalho.

Conjunto	Quantidade	% do SigmaHQ	Origem
Subconjuntos derivados do repositório SigmaHQ (4.285 regras)			
Base de conhecimento RAG (<i>rag_knowledge</i>)	2.773	64,71%	Selecionada por amostragem estratificada <i>round-robin</i> a partir do repositório SigmaHQ.
Base de teste (<i>test_cases</i>)	50	1,17%	Selecionada por amostragem estratificada <i>round-robin</i> a partir do repositório SigmaHQ.
Base de teste estendida (<i>extra_test_cases</i>)	10	0,23%	Selecionada por amostragem estratificada <i>round-robin</i> a partir do repositório SigmaHQ.
Subtotal selecionado	2.833	66,11%	—
Regras não utilizadas	1.452	33,89%	Restante do repositório, não incluído nos subconjuntos.
Regras geradas pelas LLMs comerciais (a partir de <i>test_cases</i>)			
Subconjunto	Quantidade total	Quantidade pós seleção	—
Regras geradas com <i>prompt zero-shot</i>	250	50	50 regras da <i>test_cases</i> replicadas por 5 LLMs comerciais.
Regras geradas com engenharia de <i>prompt</i>	250	50	50 regras da <i>test_cases</i> replicadas por 5 LLMs comerciais.
Regras geradas pelo agente autônomo	50	50	50 regras da <i>test_cases</i> processadas pelo agente <i>LangGraph</i> .
Subtotal de regras geradas	550	150	—

Fonte: Autoria própria (2026).

A fundação da base de dados do trabalho é o repositório oficial *SigmaHQ*. No entanto, apenas cinco subdiretórios interessam, pois são elas que contém as regras-alvo:

- *rules*;
- *rules-compliance*;

- *rules-emerging-threats*;
- *rules-placeholder*; e
- *rules-threat-hunting*.

Esta distinção é feita no código e poderá ser conferida no repositório X — mais explicações na **seção Y**.

O desenvolvimento do agente com RAG requer uma base de conhecimento (*rag_knowledge*), a qual convencionou-se por abrigar 75% da base total das regras *Sigma*. Para a elaboração das avaliações a partir das gerações das regras, optou-se por uma base de teste (*test_cases*) composta por 50 regras oficiais. E um conjunto de teste extra com outras 10 regras oficiais foi selecionado para testes repetitivos de aferição durante o desenvolvimento do agente.

Como já citado anteriormente, para confrontar a qualidade das regras geradas pelo agente desenvolvido, empregou-se a avaliação de cada regra do conjunto *test_cases* por cinco LLMs comerciais. Este arranjo de regras é segmentado em quatro grupos, sendo o primeiro composto pelas 50 regras originais do repositório oficial *SigmaHQ*, que funcionam como referência para todas as comparações subsequentes. O segundo grupo é composto pelas 250 regras-produto da replicação das cinco LLMs comerciais, utilizando *prompts zero-shot*. No grupo três, o procedimento foi repetido nas mesmas cinco LLMs, porém com *prompts* elaborados via *prompt engineering*, resultando em mais 250 regras. Por fim, o quarto grupo é composto pelas 50 regras geradas pelo agente autônomo.

Para as avaliações será realizada uma filtragem das melhores regras do grupo 2 e do grupo 3, utilizando a técnica de *LLM-as-a-judge*, o que resultará em 150 regras geradas no total, como pode ser visto na Tabela 1. Os métodos serão explicados na **seção X**.

4.1.1.1 Organização dos dados em disco

4.2 Métodos

A seguir serão apresentadas as etapas que compõem o método utilizado nesta pesquisa, detalhando cada fase do processo, desde a seleção e divisão do acervo de regras *Sigma*, o desenvolvimento do agente proposto, a composição comparativa entre as estratégias de geração investigadas e o protocolo de avaliação que será aplicado às regras geradas.

4.2.1 Seleção dos dados

A separação dos dados em conjuntos de treino e teste é um requisito metodológico fundamental para garantir a validade científica da comparação entre os quatro estágios desta pesquisa, em particular do Estágio 4. Em sistemas baseados em RAG, o risco de *data leakage*

— definido como “a introdução não intencional de informação preditiva sobre o alvo proveniente do processo de coleta, agregação ou preparação dos dados” por Kaufman *et al.* (2012), pode se manifestar de forma particularmente traiçoeira: caso a mesma regra figure simultaneamente na base recuperável pelo agente e no gabarito contra o qual sua saída será comparada (regras originais), o agente poderia, durante a etapa de *retrieval*, recuperar a própria regra de referência como contexto, invalidando as métricas de similaridade e produzindo uma superestimação artificial em seu desempenho. Esse problema é amplamente reconhecido na literatura como uma das causas mais frequentes de resultados inflados em *pipelines* de ML (Kaufman *et al.*, 2012).

Para evitar o *data leakage*, adotou-se a estratégia de bases disjuntas por construção: as regras utilizadas como gabarito de teste (conjuntos `test_cases` e `extra_test_cases`) foram fisicamente removidas do conjunto de regras disponibilizado ao RAG do agente antes da seleção deste último (`rag_knowledge`), de modo que nenhuma regra do teste pode ser recuperada como exemplo durante a geração, na execução do agente. A divisão foi implementada por meio do *script* `gerador_datasets_unificado.py`¹, desenvolvido especificamente para este trabalho. O *script* funciona assim:

O algoritmo opera de forma determinística, com a semente do gerador pseudoaleatório fixada em 42 (`random.seed(42)`), garantindo a reprodutibilidade dos experimentos: qualquer execução posterior do *script* produz exatamente a mesma partição dos dados. A varredura se restringe aos cinco subdiretórios do repositório oficial *SigmaHQ* que concentram as regras de produção (**seção base de dados**), excluindo-se de forma deliberada o diretório *deprecated* (já que contém regras descontinuadas), bem como pastas de exemplos, documentação e ferramentas auxiliares contidas no repositório.

A seleção das regras foi feita por amostragem estratificada balanceada com base no *logsource*, e não por amostragem aleatória simples: a partir de cada regra do repositório, o algoritmo deriva uma assinatura que consiste na tripla `categoria_produto_serviço`, oriunda do campo `logsource` (por exemplo, `process_creation_windows` ou `network_connection_linux`), e então agrupa as regras conforme estas categorias. Em seguida, a seleção acontece via *round-robin*, onde a ordem das categorias é inicialmente embaralhada, para eliminar algum viés de ordenação do dicionário; o algoritmo, então, percorre ciclicamente as categorias embaralhadas e, a cada passo, sorteia-se uma regra aleatória pertencente à categoria vigente, até se atingir-se a quantidade desejada (50 para o `test_cases`, por exemplo). Esse procedimento garante que os conjuntos resultantes contemplem uma maior diversidade possível de fontes de *log* dentro do conjunto amostral, com o objetivo de evitar a sobre-representação de categorias numerosas (como *Windows process creation*) que ocorreria sob amostragem aleatória simples — vale ressaltar que a aparição de mais regras *Windows* é justificável sob a lógica de que existem mais regras deste tipo no repositório original *Sigma*.

O processo produz três conjuntos disjuntos, descritos na Tabela 1 (Seção 4.1). O conjunto `test_cases`, composto por 50 regras, funciona como *baseline* de referência (gaba-

¹ Disponível em: .

rito, padrão-ouro) para os quatro estágios. O conjunto *extra_test_cases*, com 10 regras adicionais, serviu para o empreendimento de testes e tomadas de decisões durante a construção do agente. Por fim, o conjunto *rag_knowledge*, correspondente a 80% das regras restantes no *pool* disponível (calculado após a exclusão das regras de teste), é selecionado também via *round-robin* e constitui a base de conhecimento consultada pelo agente do Estágio 4 (**seção x**) durante a etapa de *retrieval*. As regras remanescentes não são utilizadas em nenhum dos estágios, ficando reservadas como margem para experimentos posteriores.

4.2.2 Estratégias de geração das regras

A avaliação proposta neste trabalho compara quatro estratégias distintas de obtenção de regras *Sigma* e se delineiam em ordem crescente de complexidade arquitetural: (i) a coleta direta de regras escritas por especialistas humanos (Estágio 1, regras de referência); (ii) geração automática por LLMs comerciais sob instrução mínima (*zero-shot*, Estágio 2); (iii) a geração pelas mesmas LLMs sob *prompts* elaborados via *prompt engineering* (Estágio 3); e (iv) a geração por um agente autônomo composto de RAG e ferramentas externas (Estágio 4). Cada estágio é descrito a seguir conforme a estrutura: objetivo, *input* ou entrada, processo e saída, de modo a facilitar a comparação direta entre eles. As regras produzidas pelos Estágios 2 e 3 são posteriormente submetidas a uma filtragem por *LLM-as-a-judge*, conforme detalhado na **Seção ??**, restando ao final 150 regras geradas para confronto com as 50 regras de referência.

4.2.2.1 Regras originais do *SigmaHQ* (Estágio 1)

O Estágio 1 não constitui propriamente uma estratégia de geração, mas a referência (*baseline*) contra a qual as demais serão comparadas. Seu objetivo é fornecer um conjunto de 50 regras *Sigma* convencionadas como corretas por construção — escritas, revisadas e mantidas pela comunidade especializada do projeto *SigmaHQ*, que servirão como gabarito para todas as métricas de avaliação descritas na **Seção ??**. Como entrada, este estágio utiliza diretamente o repositório oficial *SigmaHQ*, conforme descrito na **Seção ??**. Não há etapa de geração: o processo consiste exclusivamente na seleção determinística das 50 regras que compõem o conjunto *test_cases*, conduzida pelo *script* `gerador_datasets_unificado.py` segundo o procedimento de amostragem estratificada balanceada por *logsource* apresentado na **Seção ??**. A saída deste estágio são as 50 regras armazenadas no diretório `data/test_cases/`, cada qual em um arquivo `.yaml` nomeado segundo a convenção do repositório de origem. Essas regras cumprem dupla função no protocolo experimental: (i) servem de gabarito para o cálculo das métricas de similaridade e correspondência estrutural; e (ii) fornecem as referências externas (campos `references`, `description` e `tags`) que são extraídas e utilizadas como base para os *prompts* dos Estágios 2, 3 e 4.

4.2.2.2 Geração zero-shot com RAGs comerciais (Estágio 2)

O Estágio 2 tem como objetivo produzir regras *Sigma* a partir de LLMs comerciais sob instrução mínima, replicando o cenário no qual um analista de segurança pode recorrer a uma LLM de uso geral sem investir em elaboração de *prompts*. O paradigma adotado é o de *zero-shot prompting*, definido por Brown *et al.* (2020) como a ocasião em que “o modelo recebe apenas uma descrição em linguagem natural da tarefa, sem qualquer demonstração ou exemplo”. Trata-se da forma mais simples e direta de interação com uma LLM: o *prompt* contém apenas a instrução do que deve ser feito, cabendo ao modelo inferir, a partir de seu treinamento prévio, como executar a tarefa.

Como entrada deste estágio utiliza-se, para cada uma das 50 regras-alvo do conjunto *test_cases*, um *prompt zero-shot* (nomeado `prompt1.txt` curto e padronizado, contendo apenas o texto “Generate ONE SIGMA rule (<https://sigmahq.io/docs/basics/rules.html>) for this event:” seguido do(s) *link(s)* presentes no campo *references* da regra original — a URL entre parêntesis é uma referência crua para a LLM posicionar-se a respeito de qual regra está se falando. Não são fornecidos exemplos de regras *Sigma* previamente escritas, instruções sobre a estrutura *YAML* esperada, nem orientações específicas sobre o uso de *tags MITRE ATT&CK* ou definição de *logsource*.

O processo consiste em submeter cada um dos 50 `prompt1` para cinco LLMs comerciais distintas — *ChatGPT*, *Claude*, *DeepSeek*, *Gemini* e *Microsoft Copilot*, totalizando 250 interações independentes. Cada LLM é instruída a gerar uma única regra *Sigma* válida em formato *YAML*. A diversidade de modelos comerciais foi adotada para evitar que os resultados reflitam particularidades de uma única arquitetura ou política de treinamento. Logo, a saída deste estágio são 250 regras *Sigma*, organizadas em arquivos *YAML* dentro do diretório de cada caso de teste, junto à respectiva regra original e ao *prompt* utilizado (ver **Seção organização dados**). Essas regras compõem, juntamente com as do Estágio 3, o conjunto candidato à filtragem por *LLM-as-a-judge* descrita na **Seção ??**.

4.2.2.3 Geração com *prompt engineering* (Estágio 3)

O propósito do Estágio 3 é investigar em que medida a aplicação sistemática de *prompt engineering*, conforme conceituada na **Seção de engenharia de prompt**, eleva a qualidade das regras geradas pelas mesmas LLMs comerciais do Estágio 2. Ao contrário do paradigma *zero-shot* adotado no Estágio 2, em que a instrução é mínima e o modelo opera apenas com seu conhecimento prévio, no Estágio 3 a instrução é enriquecida com elementos que orientam o modelo quanto ao papel a ser assumido, ao formato esperado da saída e ao contexto técnico da ameaça. Como entrada deste estágio utiliza-se, para cada uma das 50 regras-alvo, um *prompt* construído a partir de um *template* padronizado no qual variam apenas as informações específicas da ameaça. Tal *template* foi elaborado com o auxílio do modelo *Claude* em processo

iterativo, enviando a regra original *Sigma* e solicitando o retorno do *prompt2*² correspondente — que incorpora cinco técnicas reconhecidas na literatura (Liu *et al.*, 2023):

- Atribuição de papel: o *prompt* inicia instruindo o modelo a operar como se fosse um “*Senior Threat Detection Engineer*”, engenheiro sênior de detecção de ameaças, em português;
- Restrição de formato de saída: a instrução determina que o modelo produza *exatamente* uma regra *Sigma* válida em YAML;
- Ancoragem por referência: inclui-se a *URL* da especificação oficial *Sigma*;
- Injeção de contexto estruturado: o conteúdo da requisição é organizado em seções nomeadas — `Threat to detect`, `Target environment` e `References`;
- Enriquecimento de domínio: além das referências externas, o *prompt* fornece descrição textual da ameaça, identificação do produto/serviço alvo e categorização de fonte de *log* esperada.

O processo de replicação se repete como no Estágio 2: cada um dos 50 *prompts* elaborados é submetido manualmente, via interface *web*, às cinco LLMs comerciais escolhidas, totalizando 250 interações independentes. Os Estágios 2 e 3 compartilham deliberadamente os mesmos modelos, regras-alvo e canal de submissão. A única variável que muda entre eles é a estratégia de *prompting*, de modo que as diferenças observadas nos resultados podem ser atribuídas exclusivamente a esse fator. Por fim, a saída deste estágio são 250 regras *Sigma*, armazenadas no mesmo diretório de cada caso de teste paralelo às regras do Estágio 2. Em conjunto com as regras do capítulo anterior, essas regras compõem o universo de 500 candidatos que serão submetidos a um processo de filtragem por *LLM-as-a-judge* descrito na **Seção de métricas**.

4.2.2.4 Geração via agente com RAG (Estágio 4)

O quarto e último estágio tem como finalidade gerar regras *Sigma* por meio do agente autônomo desenvolvido neste trabalho, configurando a estratégia de maior complexidade arquitetural dentre as quatro. Diferentemente dos Estágios 2 e 3, que recorrem a LLMs comerciais de grande porte acessadas via interface *web*, no Estágio 4 há o emprego de um modelo de linguagem aberto e de menor porte, executado localmente, cuja saída é apoiada por recuperação de contexto (RAG), consulta a fontes externas de inteligência de ameaças e validação iterativa. A descrição detalhada da arquitetura do agente, de seus componentes e funcionamento interno é apresentada na **Seção desenvolvimento do agente**; nesta seção, descreve-se apenas o papel do agente como uma das estratégias de geração avaliadas.

² Ver *prompt2* em apêndice.

Neste estágio, se utilizam, como entrada, três variantes de *prompt* para cada uma das 50 regras-alvo, identificadas como *prompt1*, *prompt2* e *prompt3*. As duas primeiras correspondem, respectivamente, ao *prompt zero-shot* do Estágio 2 e ao *prompt* com *prompt engineering* do Estágio 3; a terceira, *prompt3*, é uma versão refinada do *prompt2* em texto corrido, sem divisão em seções nomeadas. A supressão dos cabeçalhos estruturais (*Threat to detect*, *Target environment* e *References*) já é implantada no código: o agente realiza internamente, por meio de seus nós de classificação e enriquecimento, a função de identificar o tipo de entrada, consultar fontes externas e recuperar exemplos relevantes, tornando redundante a estruturação manual exigida das LLMs comerciais nos estágios anteriores. O *prompt3* é, portanto, a configuração principal do agente — desenhado especificamente para suas características arquiteturais, e é a variante adotada na comparação entre os quatro estágios. As variantes *prompt1* e *prompt2* são executadas em paralelo para subsidiar a análise complementar de sensibilidade descrita ao final desta seção. O processo de geração é totalmente automatizado e conduzido pelo *script* `sigma_agent_v17_geracao_automatica.py`³, que submete cada uma das 50 regras-alvo às três variantes de *prompt*, totalizando 150 execuções independentes do agente. Ao final de cada execução, o *script* registra um conjunto de metadados em arquivo *.csv*, contendo, para cada regra gerada: identificador do cenário, variante de *prompt* utilizada, *status* final da regra (aprovada ou reprovada pelo fluxo interno de validação sintática), número de tentativas até a interrupção, tempo total de execução, indicação de uso de busca *web*, tipo de entrada classificada pelo nó de classificação, indicador de contexto insuficiente, eventual erro de validação registrado e os caminhos dos arquivos contendo a regra produzida e o respectivo parecer do juiz semântico. Esses metadados subsidiam tanto a análise de desempenho do agente quanto a análise de sensibilidade ao *prompt*, ambas apresentadas na **Seção de resultados**.

O conjunto de saída deste estágio possui 150 regras *Sigma*, das quais 50 (geradas a partir do *prompt3*) compõem o conjunto oficial do Estágio 4 a ser confrontado com o gabarito de referência. As 100 regras restantes, geradas a partir de *prompt1* e *prompt2*, não participam da comparação entre os quatro estágios, mas são preservadas para a análise de sensibilidade entre os três tipos de *prompts*. Diferentemente dos Estágios 2 e 3, as regras do Estágio 4 não passam pela filtragem *LLM-as-a-judge* entre modelos comerciais, uma vez que o agente produz uma única regra por regra-alvo/*prompt*. Mas vale salientar que em seu fluxo interno, após a etapa de validação sintática, no Nó 6, é incorporado julgamento semântico com *LLM-as-a-judge*, descrito na **Seção desenvolvimento do agente**.

Dois eixos de análise são, portanto, articulados a partir do Estágio 4: (i) a comparação entre os quatro estágios, que confronta as 50 regras geradas pelo agente com *prompt3* contra as 100 regras dos demais estágios; e (ii) a análise de sensibilidade do agente ao *prompt*, que examina, mantidos constantes o modelo e a arquitetura, em que medida a variação da formulação da entrada (do *prompt1* ao *prompt3*) afeta a qualidade e a taxa de aprovação das

³ Disponível em:

regras geradas. O detalhamento das métricas aplicadas em cada eixo é apresentado na **Seção métricas**.

4.2.3 Desenvolvimento do agente

4.2.3.1 Visão geral da arquitetura

O agente desenvolvido neste trabalho é implementado conforme o paradigma de *graph-based agent*, no qual o fluxo de execução é representado por um grafo dirigido cujos nós correspondem a etapas funcionais distintas e as arestas correspondentes determinam a ordem e as condições de transição no fluxo. A implementação utiliza a biblioteca *LangGraph* (LangChain Team, 2024), parte do ecossistema *LangChain*, que oferece componentes fundamentais para a definição de máquinas de estados executadas sobre um estado compartilhado mutável (*GraphState*). Cada nó recebe o estado, executa sua tarefa, atualiza os campos pertinentes e o devolve ao grafo, que decide o próximo nó a ser executado com base em arestas fixas ou condicionais.

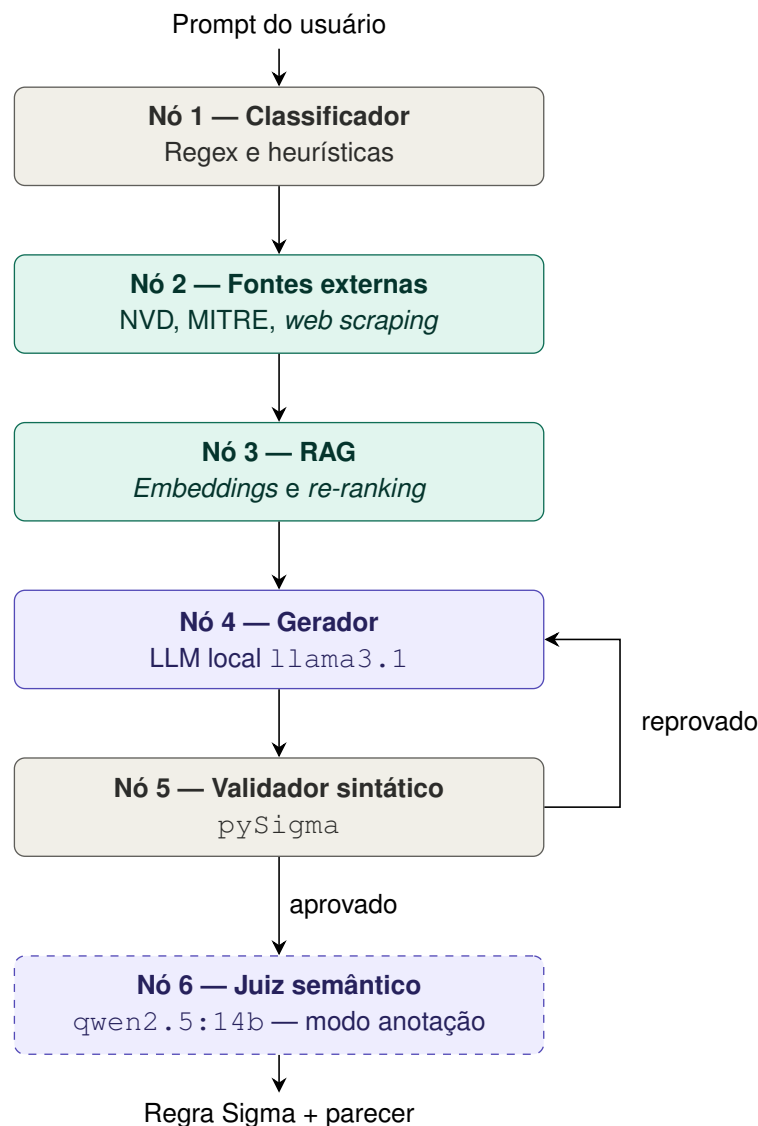
A arquitetura adotada está articulada em torno de seis nós, organizados em um *pipeline* de enriquecimento progressivo da informação. O nó 1, denominado “classificador de entrada”, é determinístico (baseado em expressões regulares e heurísticas, sem invocação de LLM) e interpreta o *prompt* recebido, identifica seu tipo (CVE, *hash* ou *texto_livre*), extrai *Uniform Resource Locators* (URL)s de referência e isola a descrição textual da ameaça. O nó 2, denominado “consulta a fontes externas”, enriquece o estado com dados técnicos obtidos junto a APIs de inteligência de ameaças (NVD, MITRE ATT&CK) e, quando necessário, recorre a *web scraping* complementado por uma sub-rotina de destilação por LLM. O nó 3, denominado “RAG”, consulta um banco vetorial contendo exemplos de regras *Sigma* previamente embarcadas, aplicando recuperação semântica por similaridade seguida de *re-ranking* por *cross-encoder*. O nó 4, denominado “geração”, invoca uma LLM local (*Llama 3.1*) que sintetiza a regra *Sigma* em YAML a partir do estado acumulado. O nó 5, denominado “validação sintática”, submete a regra gerada à ferramenta *pySigma* e emite veredito de aprovação ou reprovação. Por fim, o nó 6, denominado “juiz semântico”, opera em modo anotação — registra um parecer qualitativo sobre a regra aprovada, sem capacidade de bloqueá-la ou desencadear nova geração.

A ordem das ações adotadas pelos nós não é arbitrária. A consulta às APIs externas (nó 2) precede deliberadamente a recuperação por RAG (nó 3) para que o material técnico retornado pelas APIs (por exemplo, a descrição oficial de um CVE no NVD) sirva de consulta semântica enriquecida ao banco vetorial, aumentando a relevância dos exemplos recuperados. A geração (nó 4) ocorre apenas após o estado conter as três camadas de contexto (entrada classificada, dados de inteligência e exemplos recuperados) e é seguida imediatamente pela validação sintática (nó 5), de modo que regras malformadas sejam corrigidas antes de qualquer avaliação semântica. O grafo ainda incorpora uma aresta condicional após o nó 5: caso a regra

reprove na validação sintática, o controle retorna ao nó 4 para nova tentativa de geração, configurando um laço de auto-correção limitado por um número máximo de tentativas; caso aprove, a regra segue para o nó 6. Trata-se da única ramificação não-linear do grafo.

A Figura 3 apresenta a topologia completa da máquina de estados, com a identificação dos seis nós, suas conexões e a aresta condicional. Os seis nós são executados sequencialmente; a única ramificação não-linear é a aresta condicional após o nó 5, que retorna ao nó 4 em caso de reprovação na validação sintática. O nó 6, com borda tracejada, opera em modo anotação — registra parecer sem bloquear o fluxo.

Figura 3 – Arquitetura do agente *Sigma*



○ Determinístico ○ Contexto ○ LLM

Fonte: Autoria própria (2026).

As subseções seguintes detalham cada componente: a máquina de estados propriamente dita e o estado compartilhado (), o RAG (), as ferramentas externas (), o laço de validação sintática () e a avaliação semântica complementar ().

4.2.3.2 Máquina de estados (*LangGraph*)

Tabela 2 – Campos do *GraphState* agrupados por etapa do *pipeline*

Etapa	Campo	Descrição
Entrada bruta	<code>input_usuario</code>	<i>Prompt input</i>
Classificação (nó 1)	<code>tipo_input</code>	CVE, <i>hash</i> ou texto livre
	<code>termo_busca</code>	CVE ou <i>hash</i> extraído
	<code>url_fornecida</code>	URLs identificadas no <i>prompt</i>
	<code>texto_para_rag</code>	Descrição da ameaça pré-processada
	<code>contexto_pobre</code>	Indicador de contexto textual insuficiente
Enriquecimento (nós 2 e 3)	<code>contexto_api</code>	Dados técnicos de NVD/MITRE
	<code>contexto_rag</code>	Exemplos recuperados do banco vetorial
Geração e validação (nós 4 e 5)	<code>regra_gerada</code>	Regra <i>Sigma</i> em YAML
	<code>erro_validacao</code>	Mensagem do <i>pySigma</i> ou APROVADO
	<code>tentativas</code>	Contador do laço de correção
Avaliação semântica (nó 6)	<code>parecer_juiz</code>	Parecer do juiz em bloco JSON com cabeçalho identificador

Fonte: Autoria própria (2026).

A máquina de estados do agente é construída como um grafo dirigido sobre uma estrutura de dados única e compartilhada, chamada *GraphState*. Trata-se de um dicionário tipado em *Python* (*TypedDict*) que funciona como um “caderno de anotações” visível a todos os nós: cada nó lê os campos de que precisa, executa sua tarefa e devolve apenas os campos que atualizou (LangChain, 2024). Essa organização evita passar informações entre nós por meio de parâmetros explícitos, facilita a manutenção do agente e permite inspecionar todo o seu estado em qualquer momento da execução.

O *GraphState* possui 12 campos, que podem ser agrupados em cinco categorias de acordo com a etapa do *pipeline* em que são preenchidos, conforme a Tabela 2.

As arestas (conexões entre os nós) definem o caminho percorrido pelo estado durante a execução. Como ilustrado na Figura 3, o agente utiliza dois tipos de conexão: conexões fixas, que sempre levam ao próximo nó previamente definido, e uma única conexão condicional, onde o destino depende do conteúdo do estado naquele momento (LangChain, 2024). Cinco conexões fixas ligam, em sequência, o ponto de partida do grafo ao nó 1, e cada nó subsequente ao seguinte, até o nó 5. Após o nó 5, encontra-se a conexão condicional, cuja decisão é tomada

por uma função auxiliar chamada `roteador_de_validacao`, descrita a seguir. Por fim, uma última conexão fixa leva o nó 6 ao ponto final do grafo: o produto do agente, a regra *Sigma* (Figura 3).

A função `roteador_de_validacao` segue três regras, verificadas em ordem. Primeira: se o campo `erro_validacao` contém o valor `APROVADO`, o controle segue para o nó 6 e o laço termina com sucesso. Segunda: se o erro persiste, mas o contador `tentativas` já contabiliza 4, o controle também segue para o nó 6, encerrando o laço por esgotamento das tentativas. Entretanto, a regra considerada sintaticamente inválida ainda é gravada em disco (o *script* não a descarta nem ignora) e avaliada pelo juiz semântico, cujo parecer também é salvo para análise posterior. Terceiro: em qualquer outra situação, quando o *pySigma* é executado sobre a regra e detecta um erro, o controle volta ao nó 4 para uma nova tentativa de geração, agora orientada pela mensagem de erro guardada em `erro_validacao`. O limite de quatro tentativas é uma escolha de projeto: alto o suficiente para permitir a convergência em casos recuperáveis, mas baixo o suficiente para impedir laços infinitos diante de *prompts* que não podem ser corrigidos.

Vale destacar que essa conexão condicional é a única ramificação do grafo. Todas as outras decisões internas do agente — qual *Application Programming Interface*, em português Interface de Programação de Aplicações (API) consultar no nó 2, quantos exemplos recuperar no nó 3 e qual modelo invocar no nó 4, ocorrem dentro dos próprios nós, sem afetar a estrutura do grafo. Essa separação entre a lógica de fluxo (representada pelo grafo) e a lógica interna (escondida dentro de cada nó) é uma decisão deliberada: ela permite que cada nó seja modificado, substituído ou melhorado de forma independente, sem alterar a arquitetura geral do agente.

4.2.3.3 Ferramentas externas

4.2.3.4 RAG

4.2.3.5 Geração da regra

4.2.3.6 Validação sintática iterativa e laço de correção

4.2.3.7 Avaliação semântica por LLL-as-a-judge

4.2.4 Métricas de Avaliação

4.2.4.1 Determinísticas

4.2.4.2 Similaridade

4.2.4.3 Comparação entre estágios

5 RESULTADOS

5.1 Discussões (opcional)

5.2 Trabalhos futuros

6 CONCLUSÃO

REFERÊNCIAS

- Accenture. **Securing the Digital Economy: Reinventing the Internet for Trust**. Dublin, Ireland, 2019. Relatório corporativo. Disponível em: <https://newsroom.accenture.com/news/2019/cybercrime-could-cost-companies-us-5-2-trillion-over-next-five-years-according-to-new-research-from-accenture>
- ALCANTARA, L. *et al.* Syrius: Synthesis of rules for intrusion detectors. **IEEE Transactions on Reliability**, IEEE, v. 71, n. 1, p. 370–381, 2021.
- BANERJEE, S.; LAVIE, A. METEOR: An automatic metric for MT evaluation with improved correlation with human judgments. *In: Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization*. [S.l.: s.n.], 2005. p. 65–72.
- BOOTH, H.; RIKE, D.; WITTE, G. A. **The National Vulnerability Database (NVD): Overview**. Gaithersburg, MD, 2013.
- BROWN, T. B. *et al.* Language models are few-shot learners. *In: Advances in Neural Information Processing Systems*. [S.l.: s.n.], 2020. v. 33, p. 1877–1901.
- BUSINESS, V. **2018 Data Breach Investigations Report**. New York, USA, 2018. Disponível em: https://www.verizon.com/business/resources/reports/DBIR_2018_Report.pdf.
- BUSINESS, V. **2025 Data Breach Investigations Report (DBIR)**. USA, 2025. C. David Hylender, Philippe Langlois, Alex Pinto, Suzanne Widup (eds.). Disponível em: <https://www.verizon.com/business/resources/T437/reports/2025-dbir-data-breach-investigations-report.pdf>.
- CHOWDHARY, K. R. Natural language processing. *In: Fundamentals of Artificial Intelligence*. New Delhi: Springer, 2020. p. 603–649.
- DIMITROV, D.; NIKOLOV, D. Leveraging yara and sigma rules to detect chinese state-sponsored hacking groups of the "typhoon" type. *In: Environment. Technology. Resources. Proceedings of the 16th International Scientific and Practical Conference*. Rezekne, Latvia: RTU Press, 2025. II, p. 107–113. Disponível em: <https://doi.org/10.17770/etr2025vol2.8617>.
- DU, M.; HU, W.; HEWLETT, W. AutoCombo: Automatic malware signature generation through combination rule mining. *In: Proceedings of the 30th ACM International Conference on Information & Knowledge Management*. [S.l.]: ACM, 2021. p. 3777–3786.
- EXABEAM. **AI SIEM: How SIEM with AI/ML is Revolutionizing the SOC**. 2025. Disponível em: <https://www.exabeam.com/explainers/siem/ai-siem-how-siem-with-ai-ml-is-revolutionizing-the-soc/>. Acesso em: 20 jun. 2025.
- Forum of Incident Response and Security Teams. **Common Vulnerability Scoring System version 3.1: Specification Document**. [S.l.], 2019. Disponível em: <https://www.first.org/cvss/v3-1/specification-document>.
- GAMBLIN, J. **2025 CVE Data Review**. 2026. Análise consolidada dos dados da National Vulnerability Database (NVD): 48.185 CVEs publicados em 2025. Disponível em: <https://jerrygamblin.com/2026/01/01/2025-cve-data-review/>.
- GONZÁLEZ-GRANADILLO, G.; GONZÁLEZ-ZARZOSA, S.; DIAZ, R. Security information and event management (SIEM): Analysis, trends, and usage in critical infrastructures. **Sensors**, v. 21, n. 14, p. 4759, 2021.

HASANOV, I. *et al.* Application of large language models in cybersecurity: A systematic literature review. **IEEE Access**, IEEE, v. 12, p. 176751–176778, 2024.

HASANOV, I. *et al.* Application of large language models in cybersecurity: A systematic literature review. **IEEE Access**, IEEE, v. 12, p. 176751–176778, 2024.

IBM Security. **Cost of a Data Breach Report 2024**. Armonk, NY, 2024. Pesquisa conduzida pelo Ponemon Institute. Disponível em: <https://www.ibm.com/downloads/documents/us-en/107a02e94948f4ec>.

IBM Security. **Cost of a Data Breach Report 2025**. Armonk, NY, 2025. Pesquisa conduzida pelo Ponemon Institute. Disponível em: <https://www.ibm.com/reports/data-breach>.

JORDAN, M. I.; MITCHELL, T. M. Machine learning: Trends, perspectives, and prospects. **Science**, American Association for the Advancement of Science, v. 349, n. 6245, p. 255–260, 2015.

JURAFSKY, D.; MARTIN, J. H. **Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition**. 2. ed. Upper Saddle River, NJ: Pearson Prentice Hall, 2009. ISBN 9780131873216.

KAUFMAN, S. *et al.* Leakage in data mining: Formulation, detection, and avoidance. **ACM Transactions on Knowledge Discovery from Data**, ACM, v. 6, n. 4, p. 1–21, 2012.

LangChain. **Graph API overview — LangGraph documentation**. 2024. <https://docs.langchain.com/oss/python/langgraph/graph-api>. Acesso em: 17 jun. 2026.

LangChain Team. **LangGraph: Build resilient language agents as graphs**. 2024. <https://langchain-ai.github.io/langgraph/>. Acesso em: 10 de junho.

LEMPINEN, M.; PYYNY, E.; JUNTUNEN, A. **Chatbot for Assessing System Security with OpenAI GPT-3.5**. 2023. 34 p.

LI, J. *et al.* **GRIDAI: Generating and Repairing Intrusion Detection Rules via Collaboration among Multiple LLM-based Agents**. 2025.

LIDDY, E. D. Natural language processing. *In: Encyclopedia of Library and Information Science*. 2. ed. New York: Marcel Decker, Inc., 2001.

LIN, C.-Y. ROUGE: A package for automatic evaluation of summaries. *In: Text Summarization Branches Out: Proceedings of the ACL-04 Workshop*. [S.l.: s.n.], 2004. p. 74–81.

LIU, P. *et al.* Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. **ACM Computing Surveys**, ACM, v. 55, n. 9, p. 1–35, 2023.

Mandiant. Relatório técnico anual, **M-Trends 2026: Data, Insights, and Strategies From the Frontlines**. 2026. Tempo médio para exploração (time-to-exploit) estimado em -7 dias; exploits como vetor inicial mais comum pelo 6º ano consecutivo (32%). Disponível em: <https://cloud.google.com/blog/topics/threat-intelligence/m-trends-2026>.

MANNING, C. D.; RAGHAVAN, P.; SCHÜTZE, H. **Introduction to Information Retrieval**. Cambridge, UK: Cambridge University Press, 2008. ISBN 9780521865715.

MCGOWAN, M. **Detecting malicious activities with Sigma rules**. 2025. Disponível em: https://lantern.splunk.com/Security/UCE/Guided_Insights/Threat_hunting/Detecting_malicious_activities_with_Sigma_rules. Acesso em: 20 jun. 2025.

National Institute of Standards and Technology. **National Vulnerability Database – General Information**. 2025. Disponível em: <https://nvd.nist.gov/general>.

National Institute of Standards and Technology. **NIST Updates NVD Operations to Address Record CVE Growth**. 2026. Aumento de 263% nas submissões de CVE entre 2020 e 2025; adoção de modelo de priorização baseado em risco. Disponível em: <https://www.nist.gov/news-events/news/2026/04/nist-updates-nvd-operations-address-record-cve-growth>.

NURUSHEVA, A. *et al.* Machine learning algorithms in siem systems for enhanced detection and management of security events. *In: Bulletin of L. N. Gumilyov Eurasian National University. Mathematics, Computer Science, Mechanics Series*. Astana, Kazakhstan: L. N. Gumilyov Eurasian National University, 2024. v. 148, n. 3, p. 6–17.

PALO ALTO NETWORKS. **What Is the Role of AI and ML in Modern SIEM Solutions?** 2025. Disponível em: <https://www.paloaltonetworks.com/cyberpedia/role-of-artificial-intelligence-ai-and-machine-learning-ml-in-siem>. Acesso em: 20 jun. 2025.

PAPINENI, K. *et al.* BLEU: a method for automatic evaluation of machine translation. *In: Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL)*. [S.l.: s.n.], 2002. p. 311–318.

PICUS SECURITY. **What Is a YARA Rule?** 2025. Disponível em: <https://www.picussecurity.com/resource/glossary/what-is-a-yara-rule>. Acesso em: 20 jun. 2025.

PODZINS, O.; ROMANOV, A. Why siem is irreplaceable in a secure it environment? *In: 2019 Open Conference of Electrical, Electronic and Information Sciences (eStream)*. [S.l.: s.n.], 2019. p. 1–5.

Rapid7 Labs. Relatório técnico anual, **The 2026 Global Threat Landscape Report: Decoding the Accelerated Cyber Attack Cycle**. 2026. Mediana entre divulgação de vulnerabilidade e inclusão no catálogo CISA KEV reduzida de 8,5 para 5,0 dias; exploração confirmada de vulnerabilidades CVSS 7-10 cresceu 105% (de 71 em 2024 para 146 em 2025). Disponível em: <https://www.rapid7.com/research/report/global-threat-landscape-report-2026/>.

ROTH, F. **Rule Creation Guide**. 2022. Disponível em: <https://github.com/SigmaHQ/sigma/wiki/Rule-Creation-Guide>. Acesso em: 20 jun. 2025.

ROY, S. *et al.* **SoK: The MITRE ATT&CK Framework in Research and Practice**. 2023. arXiv:2304.07411 [cs.CR]. Disponível em: <https://arxiv.org/abs/2304.07411>.

SADDI, V. R. *et al.* Examine the role of generative ai in enhancing threat intelligence and cyber security measures. *In: Proceedings of the 2024 2nd International Conference on Disruptive Technologies (ICDT)*. India: IEEE, 2024. p. 537–542.

SANS Institute. White paper, **SANS 2025 Detection and Response Survey**. 2025. Survey anual independente e vendor-neutral (3ª edição), conduzido por Josh Lemon. 73% das organizações apontam falsos positivos como o principal desafio de detecção; 76% citam fadiga de alertas como preocupação central do SOC. Disponível em: <https://www.sans.org/white-papers/2025-detection-response-survey-webcast-forum>.

SCHWARTZ, Y. *et al.* LLMCloudHunter: Harnessing LLMs for automated extraction of detection rules from cloud-based CTI. *In: Proceedings of the ACM Web Conference 2025 (WWW '25)*. Sydney, NSW, Australia: ACM, 2025. p. 1922–1941. ISBN 979-8-4007-1274-6/25/04.

SCHWARTZ, Y. *et al.* **LLMCloudHunter: Harnessing LLMs for Automated Extraction of Detection Rules from Cloud-Based CTI**. 2024.

SHAH, A. H. *et al.* Automated log analysis and anomaly detection using machine learning. *In: Fuzzy Systems and Data Mining VIII*. Amsterdam, Netherlands: IOS Press, 2022, (Frontiers in Artificial Intelligence and Applications, v. 358). p. 137–147.

SHEERAZ, M. *et al.* Revolutionizing SIEM security: An innovative correlation engine design for multi-layered attack detection. **Sensors**, v. 24, n. 15, p. 4901, 2024.

SHEPARDSON, D. **SMarriott CEO apologizes for data breach, unsure if China responsible**. 2019. Disponível em: <https://www.reuters.com/article/us-usa-cyber-congress/marriott-ceo-apologizes-for-data-breach-unsure-if-china-responsible-idUSKCN1QO217/>. Acesso em: 20 jun. 2025.

SIGMAHQ. **About Sigma**. 2025. Disponível em: <https://sigmahq.io/docs/guide/about.html>. Acesso em: 20 jun. 2025.

SIGMAHQ. **Sigma - Generic Signature Format for SIEM Systems**. 2025. Disponível em: <https://github.com/SigmaHQ/sigma/tree/master>. Acesso em: 20 jun. 2025.

SIGMAHQ. **Sigma Rules**. 2025. Disponível em: <https://sigmahq.io/docs/basics/rules.html>. Acesso em: 20 jun. 2025.

STROM, B. E. *et al.* **MITRE ATT&CK: Design and Philosophy**. [S.l.], 2018. Revised March 2020.

TARIQ, S. *et al.* Alert fatigue in security operations centres: Research challenges and opportunities. **ACM Computing Surveys**, v. 57, n. 9, p. 1–38, 2025.

TENDIKOV, N. *et al.* Security information event management data acquisition and analysis methods with machine learning principles. **Results in Engineering**, v. 22, p. 102254, 2024.

The MITRE Corporation. **CVE Program Celebrates 25 Years of Impact in Cybersecurity**. 2024. Disponível em: <https://www.mitre.org/news-insights/news-release/cve-program-celebrates-25-years-impact-cybersecurity>.

The MITRE Corporation. **About CWE – Common Weakness Enumeration Overview**. 2025. Disponível em: <https://cwe.mitre.org/about/index.html>.

The MITRE Corporation. **CVE List Basics: Identifier Syntax and Numbering Authorities**. 2025. Disponível em: <https://www.cve.org/ResourcesSupport/AllResources/CNARules>.

VASWANI, A. *et al.* Attention is all you need. **Advances in Neural Information Processing Systems**, v. 30, 2017.

VIELBERTH, M. *et al.* Security operations center: A systematic study and open challenges. **IEEE Access**, v. 8, p. 227756–227779, 2020.

WANG, H. *et al.* RulePilot: An LLM-powered agent for security rule generation. *In: Proceedings of the 2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE '26)*. Rio de Janeiro, Brazil: ACM, 2026.

Wazuh, Inc. **Wazuh - The Open Source Security Platform**. 2024. <https://wazuh.com/>. Copyright © 2015-2024 Wazuh, Inc.; Accessed: 26 June 2025.

WEI, J. *et al.* Chain-of-thought prompting elicits reasoning in large language models.
In: Advances in Neural Information Processing Systems. [S.l.: s.n.], 2022. v. 35, p.
24824–24837.

GLOSSÁRIO

BeautifulSoup . 26

DeepSeek . 28

F1 . 29

GLM . 30

GPT . 26, 28–30

LangGraph . 32

LLaMA . 28

MITRE . 16

SigmaHQ . 32

Snort . 29

Splunk . 27, 28

Suricata . 29

Threat Hunting . 12

YAML . 41

zero-day . 8, 9, 14

zero-shot . 33, 34